Reasonless Computation

Manifold-Aligned Generative Inference,
Computational Faithfulness,
and the Limits of Statistical Authority

Flyxion
Independent Researcher

October 2026

Abstract

Generative systems that emit intermediate tokens before a final answer are frequently described as deliberating, planning, or executing algorithms, and the success of their final answers is routinely taken as evidence for those descriptions. Recent experiments on maze-solving transformers trained on valid and deliberately mismatched search traces show that final-answer accuracy can be high while the emitted trace fails the semantics of the algorithm it appears to execute. This monograph takes that dissociation as an empirical starting point and asks two further questions. The first concerns mechanism: if the apparent meaning of intermediate tokens is not what makes them useful, what structure in the underlying computation is being exploited. The second concerns authority: whatever the answer to the first question, what additional warrant is needed before the output of a generative system may be executed as an action.

To address the first question the monograph develops a geometric account of generative computation based on manifold-aligned inference, in which meaningful updates are constrained to the tangent structure of a learned manifold of coherent states, and it separates the textual faithfulness of a trace from the geometric coherence of the internal trajectory the trace induces. To address the second it separates computational utility, procedural validity, causal faithfulness, geometric coherence, outcome correctness, and policy admissibility into independent evaluative domains, proves that none of the first five entails the sixth, and states and proves an execution boundary proposition under explicit assumptions of complete mediation, atomic validation, and verifier soundness. The final part proposes a reproducible experimental program, including a test of whether geometric properties of internal trajectories explain variation in task success beyond that explained by trace validity, and an evaluation of architectures that differ in who holds the authority to execute.

Throughout, the monograph distinguishes what is established empirically by the sources it uses, what follows logically from its definitions, what is conjectured, and what must be guaranteed architecturally. The formal results are consequences of definitions and stated assumptions and are presented as such. The experimental proposals are designs to be carried out, not results already obtained.

Contents

Abstract **iii**

I Conceptual Foundations **1**

1 Introduction: When Success Becomes Authority **3**
1.1 The Apparent Success of Intermediate Reasoning 3
1.2 The Experimental Disruption 3
1.3 The Second Promotion 4
1.4 The Thesis of Non-Entailment 4
1.5 A Positive Account and a Boundary 4
1.6 Contributions and Scope 5
1.7 Terminology and Conventions 5
1.8 Three Registers of Claim 6
1.9 Why the Question Is Not Merely Terminological 6

2 Six Properties Mistaken for One **9**
2.1 The Evaluative Dimensions 9
2.2 Computational Utility 9
2.3 Procedural Validity 10
2.4 Causal Faithfulness 10
2.5 Geometric Coherence 10
2.6 Outcome Correctness and Policy Admissibility 10
2.7 Independence and Linking Assumptions 11
2.8 The Maze Domain Under the Six Predicates 11
2.9 How Each Property Is Measured 12
2.10 Why the Properties Should Not Be Collapsed 12

3 The Epistemology of Generated Explanations **13**
3.1 Five Kinds of Account 13
3.2 Representation Does Not Establish Occurrence 13
3.3 Declarations and Inspectable Evidence 14
3.4 Rationalization, Mechanism, Proof, and Operation 14
3.5 Against Two Easy Responses 14
3.6 Self-Report in People and in Models 14
3.7 A Ladder of Evidence 15
3.8 What the Ladder Does Not Settle 15

4 Statistical Authority and Its Illegitimate Extensions **17**
4.1 Descriptive and Normative Authority 17
4.2 Sources of Inflated Authority 17
4.3 Authority Laundering 17
4.4 The Parallel with Trace Validity 18
4.5 A Typology of Promotions 18
4.6 Calibration and What It Licenses 18
4.7 Authority Under Distribution Shift 19
4.8 Self-Generated Evidence 19

II The Geometric Substrate **21**

5 Manifold-Aligned Generative Inference **23**
5.1 The Original Argument 23
5.2 What the Criterion Does and Does Not Say 24
5.3 From Generation to Computation 24
5.4 The Tangent-Normal Decomposition in Coordinates 24
5.5 Noise Models and the Conditions Under Which Normal Variation Dominates 25
5.6 Prediction Targets and What They Imply 25

6 Potentials, Stratification, and Descent **27**
6.1 Gradient Flow and Descent 27
6.2 Morse Structure 27
6.3 Stratified State Spaces 28
6.4 Three Conditions Kept Apart 28
6.5 Gradient Flow in Latent Coordinates 28
6.6 Gradient Flows Do Not Cycle 28
6.7 Discrete Updates and Retractions 29

7 Hidden States as Approximate Manifolds **31**
7.1 The Representation Problem 31
7.2 Choosing a Representation 31
7.3 Local Estimators and Off-Manifold Residuals 31
7.4 Relating Token Interventions to State Transitions 32
7.5 Curvature and the Bias of Secant Estimates 32
7.6 Choosing the Dimension and the Layer 33

III Reasonless Intermediate Tokens **35**

8 Formal Reconstruction of the Source Experiments **37**
8.1 The Domain 37
8.2 Three Training Regimes 37
8.3 Reported Findings 38
8.4 The Precise Conclusion and Its Limits 38
8.5 What the Validator Checks 38

8.6 The Swapped Regime as an Intervention on Supervision 39
8.7 Reading Cross-Distribution Results 39
8.8 Threats to Inference 39

9 Information, Computation, and Intermediate Tokens 41
9.1 Problem Information and Computational Transformation 41
9.2 Training Traces and Generated Traces 41
9.3 Extra Tokens as Computational Opportunity 41
9.4 The Serial Depth Argument 42
9.5 Exposure and Self-Conditioning 42
9.6 Entropy of Generated Traces 43

10 The Computational Scaffolding Hypothesis 45
10.1 Statement 45
10.2 An Interventional Measure of Utility 45
10.3 What Scaffolds Might Depend On 45
10.4 Proposed Comparisons 46
10.5 A Taxonomy of Mechanisms 46
10.6 Relation to Soft Prompts and Learned Prefixes 47
10.7 Statistical Considerations 47

11 The Generalization Paradox 49
11.1 The Observation 49
11.2 Competing Explanations 49
11.3 Competence Versus Task Completion 49
11.4 An Experimental Design 50
11.5 A Stylized Model of the Non-Monotone Pattern 50
11.6 What Each Explanation Predicts 50

12 Trace Length and the Appearance of Adaptive Deliberation 53
12.1 The Comparison 53
12.2 Measures of Association 53
12.3 Difficulty Is Relative to an Algorithm 53
12.4 Four Distinct Quantities 54
12.5 Tests for Adaptive Allocation 54
12.6 Censoring Formalized 54
12.7 Confounding by Input Size 54
12.8 Role in the Argument 55

IV Faithfulness and Non-Entailment 57

13 A Logic of Evaluative Independence 59
13.1 The Formal Universe 59
13.2 The Independence Proposition 59
13.3 Conditional Closure: Which Implications Can Be Recovered 60
13.4 Independence as a Design Constraint 61

14 The Limits of Trace Verification **63**
14.1 Three Levels . . . 63
14.2 A Verifier as a Predicate . . . 63
14.3 A Taxonomy of False Inferences . . . 63
14.4 On Verifying the Mechanism Itself . . . 64
14.5 Ordering Specifications by Strength . . . 64
14.6 Soundness and Completeness of a Specification . . . 64
14.7 Process Supervision and Its Verifier . . . 65

15 Causal Faithfulness and Intervention **67**
15.1 A Causal Model of Generation . . . 67
15.2 Three Criteria . . . 67
15.3 Influence Without Semantic Faithfulness, and the Reverse . . . 68
15.4 Limits of Inference from Interventions . . . 68
15.5 A Catalogue of Interventions . . . 68
15.6 Estimands . . . 69
15.7 Identification . . . 69

16 Geometric Faithfulness and Textual Faithfulness **71**
16.1 Two Faithfulness Questions . . . 71
16.2 The Four Cells . . . 71
16.3 The Geometric Faithfulness Hypothesis . . . 72
16.4 What Success Would and Would Not Show . . . 72
16.5 Alternative Readings of an Association . . . 72
16.6 Interventions on the Trajectory . . . 72

V Admissibility and the Execution Boundary **75**

17 The Non-Entailment of Permission **77**
17.1 From Answers to Actions . . . 77
17.2 The Proposition . . . 77
17.3 Illustrations . . . 77
17.4 Possibility and Permission . . . 78
17.5 The Structure of Policies . . . 78
17.6 Technical Possibility and Normative Permission . . . 78
17.7 Delegation and Standing . . . 79
17.8 When a Policy May Make a Property Sufficient . . . 79

18 Manifold Coherence Is Not Admissibility **81**
18.1 A Tangent Update Can Leave the Admissible Region . . . 81
18.2 A Constrained Update . . . 81
18.3 The Role of the Construction . . . 82
18.4 A Worked Example on the Circle . . . 82
18.5 Retraction and Finite Steps . . . 83
18.6 When the Linearization Fails . . . 83

19 The Proposal, Verification, and Execution Architecture **85**
19.1 Four Roles . . . 85
19.2 What the Decision May Depend On . . . 85
19.3 Independence of the Roles . . . 85
19.4 The Trusted Computing Base . . . 86
19.5 Information Flow Between the Roles . . . 86
19.6 What the Generator May Observe . . . 87

20 Soundness of the Execution Boundary **89**
20.1 The Invariant . . . 89
20.2 Assumptions . . . 89
20.3 The Proposition . . . 89
20.4 Status of the Result . . . 90
20.5 Bounded Exposure Under Mediation . . . 90
20.6 Failure of the Assumptions . . . 91
20.7 Proof Obligations in an Implementation . . . 91
20.8 Composition of Guards . . . 92
20.9 Soundness Without Usefulness . . . 92

21 Refusal, Uncertainty, and Incomplete Authorization **93**
21.1 Beyond Permitted and Forbidden . . . 93
21.2 Conservative Semantics . . . 93
21.3 Refusal and Impermissibility . . . 93
21.4 Propagation Through Plans . . . 94
21.5 Reason Codes and Their Uses . . . 94
21.6 Escalation . . . 94
21.7 The Cost Structure of Conservatism . . . 94

22 Authority Laundering Through Tool Chains **97**
22.1 How Explanations Acquire Operational Force . . . 97
22.2 A Provenance-Sensitive Model . . . 97
22.3 Legitimate Delegation . . . 98
22.4 Limits of Label Propagation . . . 98
22.5 Illustrative Chains . . . 98
22.6 Taint Tracking and Capability Tokens . . . 99
22.7 Endorsement as an Event . . . 99

23 Historical Accountability and Deterministic Replay **101**
23.1 Append-Only Histories . . . 101
23.2 Account Versus Record . . . 101
23.3 Operations in the SpherePOP Setting . . . 101
23.4 Replay Is Not Permission . . . 102
23.5 Log Design . . . 102
23.6 Privacy and Redaction . . . 102
23.7 Replay Under Nondeterministic Effects . . . 103

VI Optimization, Distillation, and Institutional Design 105

24 Reward Optimization and Procedural Deterioration 107
24.1 The Reported Pattern . . . 107
24.2 Outcome Measures and Procedural Invariants . . . 107
24.3 Four Ways of Handling a Constraint . . . 107
24.4 Goodhart-Style Effects . . . 108
24.5 Constrained Optimization and Its Guarantees . . . 108
24.6 Process Rewards and Their Limits . . . 109
24.7 Indifference of the Reward . . . 109

25 Distillation Without Semantic Fidelity 111
25.1 Four Forms of Transfer . . . 111
25.2 Competence Without Explanation . . . 111
25.3 Proposed Experiments . . . 111
25.4 A Measurement Matrix for Compression . . . 112

26 Verification as an Independent Institution 113
26.1 The Principle . . . 113
26.2 Comparison Across Domains . . . 113
26.3 Independent Evidence and Independent-Looking Reports . . . 113
26.4 Costs of Verification . . . 114
26.5 Correlated Failure of Checkers . . . 114
26.6 Sources of Diversity . . . 114

VII The Empirical Program and Conclusions 115

27 The Experimental Program 117
27.1 Status of the Proposals . . . 117
27.2 Experiment I: Geometric Residuals and Task Success . . . 117
27.3 Experiment II: Scaffold Utility . . . 118
27.4 Experiment III: Admissible Agents . . . 118
27.5 Predictions . . . 118
27.6 Analysis Plan and Threats to Validity . . . 119
27.7 What Would Change the Conclusions . . . 119

28 Scope and Limits 121
28.1 Formal and Open-Ended Domains . . . 121
28.2 Verification That Uses Language Models . . . 121
28.3 Stronger Models . . . 121
28.4 Limits of the Geometric Account . . . 122
28.5 Open Problems . . . 122

29 Conclusion: Reasons Are Not Permissions 123

A Formal Language and Notation 125

B Countermodels **127**

C Execution Boundary Soundness: Formal Model **129**

D Experimental Protocols **131**

E Replication Status **133**

F Execution Histories in the SpherePOP Setting **135**

G Limitations and Failure Cases **137**

Bibliography **139**

Part I

Conceptual Foundations

Chapter 1

Introduction: When Success Becomes Authority

1.1 The Apparent Success of Intermediate Reasoning

The practice of having a generative model emit intermediate text before committing to a final answer has become one of the organizing techniques of contemporary machine learning. It appears in several forms that are often discussed together although they differ in mechanism: prompting a model to produce a chain of thought [28], training on supervised scratchpads that record intermediate computation [19], and post-training with reinforcement learning against verifiable rewards, in which the intermediate text is shaped only indirectly by the success of the answer that follows it [23]. In all three cases the observed pattern is that answers produced after intermediate text are, on many tasks, more accurate than answers produced without it. That pattern has encouraged a reading in which the intermediate text is evidence of deliberation, planning, or the stepwise execution of an algorithm. The reading is natural because the text is written in a register that describes such activities, and because human problem solvers who write out their steps usually do so because the steps are the means by which they reach the answer.

The reading nevertheless contains an inference that is rarely stated. It moves from the fact that intermediate text improves performance, to the claim that the text means what it appears to mean, to the claim that the computation inside the model corresponds to what the text says. Each step requires evidence that the preceding step does not supply. This monograph is concerned with the structure of that missing evidence, and with a second inference of the same shape that arises when generative systems are given the ability to act.

1.2 The Experimental Disruption

The empirical starting point is the study of Valmeekam et al. [26], which examines transformers trained to solve mazes while emitting traces of the A* search algorithm. The study compares models trained on solutions only, models trained on correct A* traces, and models trained on traces that were deliberately swapped between problem instances so that each trace is a well formed A* trace for some other maze. The reported finding that organizes this monograph is that models trained on swapped traces can reach high final-answer accuracy, in some evaluation distributions higher than models trained on

correct traces, while the traces they generate do not satisfy the semantics of A* execution. The study further reports that in some post-training experiments with group relative policy optimization, answer accuracy improved while measured trace validity deteriorated. Chapter 8 reconstructs the experimental design in the formal terms needed here and states precisely what the reported results do and do not establish.

The precise empirical conclusion is narrow. In the evaluated settings, high final-answer accuracy does not require the model to generate a trace that is valid under the prescribed execution semantics of A*. The finding does not establish that the models performed no meaningful internal computation, that intermediate tokens are generally useless, or that trace validity is irrelevant in other settings. It establishes a dissociation between two properties that were commonly assumed to travel together.

1.3 The Second Promotion

A parallel inference arises when a generative model is embedded in a system that can act. A model whose predictions have been accurate, whose explanations are coherent, and whose stated confidence is well calibrated on past cases comes to be treated as entitled to act on its conclusions: to call a tool, to modify a record, to spend a resource, to send a message. The inference runs from statistical performance to permission, and it has the same logical form as the inference from answer accuracy to trace validity. In both cases a property established by evidence about outcomes is silently promoted to a property of a different kind. The second promotion is the more consequential because its error is not merely a mistaken description of a computation but an action taken in the world without the warrant that the action required.

1.4 The Thesis of Non-Entailment

The central claim of the monograph is that a family of properties which are usually run together are logically and empirically independent. A generated output can be correct while its accompanying explanation is not a valid derivation. An explanation can be a valid derivation of the output while having played no causal role in producing it. A computation can proceed along a coherent internal trajectory while its emitted text describes something else. A correct, valid, and faithful computation can propose an action that no applicable authority has permitted. Each of these separations is developed in the chapters that follow, and each is accompanied by a statement of the additional assumption that would be required to close the gap, so that the reader can see what is being assumed whenever a gap is closed in practice.

1.5 A Positive Account and a Boundary

A purely negative thesis, that intermediate tokens do not mean what they appear to mean, leaves open what they are doing. The monograph therefore develops a candidate positive account, built on earlier work on manifold-aligned generative inference [5, 6]. On this account the usefulness of a generated sequence is to be sought in the representational dynamics it induces rather than in the procedure it describes, and the relevant notion of coherence is

geometric. This account is offered as a hypothesis that can be formalized and tested, not as an established explanation, and Part II is careful about the conditions under which the geometric machinery applies to the hidden states of an autoregressive model at all.

The second half of the argument concerns the boundary between proposal and execution. It extends the account of the execution boundary developed in [8], which states mediated execution, the non-entailment of permission by justification, and a bound on exposure for single actions, from the single-action setting to a framework of six evaluative properties. It builds on the separation of reasons from permissions developed there and gives that separation a formal statement, an execution boundary proposition, and an experimental evaluation.

1.6 Contributions and Scope

The contributions are of four kinds, and the monograph marks each as such. The empirical observations come from the source experiments and are reported as findings of that study, not as results of this one. The formal propositions, in particular the independence proposition of Chapter 13, the non-entailment of permission in Chapter 17, the admissible projection of Chapter 18, and the execution boundary proposition of Chapter 20, are consequences of definitions and explicit assumptions. The hypotheses, especially the geometric-faithfulness hypothesis of Chapter 16, are conjectures with stated tests. The architectural proposals describe design rules whose guarantees hold only under the stated assumptions. Chapter 28 returns to the question of which of these claims survive outside formal domains with decidable specifications.

The length and organization follow the argument. Part I sets out the conceptual framework and the six evaluative properties. Part II recovers the geometric account and states its limits. Part III reconstructs the experimental dissociation. Part IV proves non-entailment among the properties of answer generation and develops the distinction between textual and geometric faithfulness. Part V extends the analysis to action and states the execution boundary proposition. Part VI considers optimization, distillation, and verification as institutions. Part VII gives the experimental program and the conclusions.

1.7 Terminology and Conventions

Several words in this literature carry more meaning than the evidence supports, and the monograph fixes its usage at the outset. A task is the input given to a generative system, an intermediate sequence or trace is the text the system emits before committing to its final output, and the answer or output is what follows. A proposal is an output that has the form of an action, such as a tool call, a record modification, or a message to a third party. The word reasoning is used descriptively, to name the practice of emitting intermediate text, and it carries no commitment about whether the underlying computation deserves the name. The word valid is always relative to a stated specification, and a statement that a trace is valid without a specification is treated as incomplete. The word faithful is used only for the interventional notion defined in Chapter 2. The word authority is used in two senses, descriptive and normative, which are separated in Chapter 4, and the monograph never uses it without indicating which is meant.

These conventions are not pedantic. Much of the apparent disagreement about whether language models reason arises from a shift in the sense of a word within a single argument. A claim that a trace is valid, offered as evidence that the model followed the procedure, uses valid in a textual sense in its premise and in a mechanistic sense in its conclusion. A claim that a model is trusted because it has been reliable uses trust in a statistical sense in its premise and in a normative sense in its conclusion. Holding the senses fixed makes such shifts visible.

1.8 Three Registers of Claim

The monograph makes claims in three registers and keeps them apart, because the grounds on which a claim can be accepted differ between them. Empirical claims concern what was observed in a stated setting, and they are accepted on the strength of the experiment that produced them, including its design, its sample, and its controls. The empirical claims used here come from the source study and are attributed to it. Logical claims concern what follows from definitions and stated assumptions, and they are accepted on the strength of a proof, which the reader can check without trusting any experiment. The independence proposition and the execution boundary proposition are of this kind. Architectural claims concern what a system will do because it has been built in a particular way, and they are accepted on the strength of an argument that the construction enforces the property, together with an account of which assumptions the argument uses.

A fourth category, conjecture, is used for the proposals of the geometric account and the experimental predictions. A conjecture is labeled as such, comes with a statement of what would refute it, and is not used as a premise in any logical or architectural claim. This discipline is the minimal defence against the error the monograph describes. A document that argued against promoting evidence of one kind into a conclusion of another, while itself promoting a conjecture into a premise, would commit the error in its own construction.

1.9 Why the Question Is Not Merely Terminological

It might be objected that the question of whether intermediate tokens are reasoning is a dispute about a word, and that nothing of practical consequence turns on how it is resolved. Three practical matters show otherwise. The first is oversight. Developers and operators increasingly read generated traces as a means of monitoring what a system is doing, and the value of that practice depends on the trace being a reliable indicator of the computation. If the trace is a scaffold whose content does not describe the computation, a monitor that reads it for evidence of misbehavior may find none where there is some, and may find alarming content where there is none.

The second is training. Process supervision rewards intermediate steps that a verifier judges correct, on the assumption that rewarding correct steps produces a system that reasons correctly. If the usefulness of the steps lies elsewhere, process supervision may shape the surface of the trace without shaping the computation, or may shape the computation in unintended ways. The third is deployment. A system whose proposals are executed on the strength of its record of correct outputs, without a separate authorization step, is exposed to every proposal that is correct in the sense of the task and forbidden in the sense of the

policy. Each of these is a decision with consequences, and each is taken on the basis of an interpretation of what intermediate text is. The interpretation is therefore a matter for evidence and not for convention.

Chapter 2

Six Properties Mistaken for One

2.1 The Evaluative Dimensions

Let x range over tasks, z over intermediate sequences, y over final outputs, and a over proposed actions. Six properties are distinguished. Computational utility $U(x, z)$ records whether the inclusion of z improves a specified performance measure relative to a counterfactual baseline. Procedural validity $V(x, z, y)$ records whether the triple conforms to an explicitly specified grammar, transition system, and acceptance criterion. Causal faithfulness $F(x, z, y)$ records whether z accurately represents causally relevant aspects of the process that produced y, under a stated account of representation and intervention. Geometric coherence $\Gamma(x, z)$ records whether the internal trajectory induced by z satisfies a specified condition of alignment with a learned structure of task-relevant states. Outcome correctness $C(x, y)$ records whether y satisfies an externally specified task criterion, independently of how it was obtained. Policy admissibility $A_\Pi(x, a)$ records whether the action a satisfies the constraints imposed by an applicable authority, environment, and execution policy Π.

The sixth property differs from the others in its argument list. The first five concern the relation between a task, a generated sequence, and an output. Admissibility concerns an action and a policy, and the policy is not a function of the model. This difference in kind is the reason that no amount of progress on the first five can discharge the sixth.

2.2 Computational Utility

An intermediate sequence is useful if its inclusion improves a specified performance measure relative to a defined counterfactual baseline. The definition has three parameters that must be fixed before the word useful has content: the performance measure, the counterfactual, and the population of tasks over which the improvement is averaged. A sequence may be useful on one population and harmful on another, and may be useful relative to the absence of any sequence but not relative to a sequence of random tokens of equal length. The definition is intentionally silent about why the sequence helps. Utility is an interventional and comparative property, and Chapter 10 gives it a precise form.

2.3 Procedural Validity

A trace is valid relative to a specification $\mathcal{S}$ consisting of a trace language, a state-transition system, and an acceptance criterion. For the maze domain, $\mathcal{S}$ is the execution semantics of A*, and validity means that the sequence of expansions and queue operations recorded in the trace is the sequence the algorithm would perform on the given maze. Validity is a property of text relative to a specification. It makes no reference to the process that generated the text. A trace copied from a reference implementation is valid, and so is a trace produced by a model with no internal search at all that happens to produce the correct text.

2.4 Causal Faithfulness

An explanation is faithful when it accurately represents the factors that causally contributed to the result. The definition requires a theory of what it is for a linguistic object to represent a causal factor, and a protocol of intervention by which the representation is tested. Faithfulness is therefore not binary in practice. It is established relative to a family of interventions, such as truncation or replacement of the visible trace, or manipulation of internal representations, and relative to a standard for the match between what the text claims and how the output changes under intervention. Empirical studies of chain-of-thought faithfulness [25, 13] are best read as establishing specific failures of faithfulness under specific interventions, not as measuring a single quantity.

2.5 Geometric Coherence

Geometric coherence is introduced here because the positive account of Part II requires it and because it cannot be reduced to either validity or textual faithfulness. It asks whether the sequence of internal states traversed during generation moves along directions supported by a structure of task-relevant states. Its precise definition depends on a representation, a manifold estimator, and a tolerance, and Chapter 7 states these dependencies. For present purposes it suffices to note that coherence concerns a trajectory, validity concerns a text, and the relation between them is exactly what is at issue.

2.6 Outcome Correctness and Policy Admissibility

Outcome correctness is the least controversial of the properties and the one most easily mistaken for the others. A result satisfies an externally specified criterion regardless of how it was obtained. Policy admissibility is of a different logical type. It is satisfied by a pair of a situation and an action, relative to a policy, and the policy encodes who is entitled to authorize what under which conditions. Two actions with identical predicted consequences can differ in admissibility because they are taken by different principals, on different data, or under different delegations.

Premise	Conclusion	Linking assumption required
Correctness C	Validity V	The only way to produce a correct output is to execute the specified procedure, so the trace is a record of that execution.
Validity V	Causal faithfulness F	The output is a function of the trace alone, and the trace is not post hoc.
Utility U	Validity V	The usefulness of the sequence is mediated by its procedural semantics and by nothing else.
Utility U	Geometric coherence Γ	The representation and estimator are such that useful trajectories are exactly those with small off-manifold residual.
Validity V	Geometric coherence Γ	Procedurally valid text induces on-manifold internal updates.
Any of the above	Admissibility A_Π	The policy Π explicitly makes the property sufficient for permission.

Table 2.1: Linking assumptions required to obtain each commonly assumed implication. None of the implications holds without its assumption.

2.7 Independence and Linking Assumptions

The relationships among the properties fall into three classes that should not be confused: logical entailments, empirical correlations, and architectural guarantees. A logical entailment holds in every model of the definitions. An empirical correlation holds in a population of systems and tasks and may fail in another. An architectural guarantee holds because a system has been built so that a property is enforced by a mechanism, and it holds exactly as long as the assumptions about the mechanism hold. Table 2.1 lists for each pair of properties that are commonly run together the assumption that would have to be added to obtain the implication.

2.8 The Maze Domain Under the Six Predicates

The six properties can be illustrated on the maze domain of the source experiments, where each of them has a concrete reading. A task is a maze with a start and a goal. Utility is the improvement in the probability of producing a correct path when a given intermediate sequence precedes the answer, compared with a stated baseline such as an empty prefix. Validity is conformity of the emitted trace to the execution semantics of A* on that maze, as checked by a formal validator. Faithfulness is the extent to which changes to the trace in the manner the trace claims, for instance changing which cell is recorded as expanded, produce the corresponding changes in the answer. Geometric coherence is the extent to which the hidden states traversed while the trace is produced stay close to the estimated manifold of states reached on successful solutions. Correctness is the property that the emitted path leads from start to goal without crossing a wall, and, if optimality is required, that it is shortest. Admissibility is, in a deployment in which the maze solver controls a robot, the property that the route does not enter a region the operator has excluded.

Taken together these show how a single run can sit at different values on the six properties.

A model can produce a shortest path, which is correct, with a trace that expands cells in an order A* would not use, which is invalid, because it has internally computed the path by a different route. Its trace may nevertheless alter the answer when edited, so that it is influential, and its hidden states may be well inside the region reached by successful runs, so that it is coherent. The path may cross a region the operator has excluded, so that the action of following it is inadmissible. Every one of these combinations can occur, and the maze domain makes them easy to state and, for the first four, to measure.

2.9 How Each Property Is Measured

Each property is established by a different instrument, with a different kind of error, and the differences matter when results are compared across studies. Utility is estimated by comparing success rates under interventions on the prefix, and its error is sampling error in the success rates together with the error arising from differences between the intervention distribution and the natural distribution. Validity is established by a deterministic validator, and its error is the error of the specification, in the sense that the validator may encode a convention such as a tie-breaking rule that the model has not been trained to respect. Faithfulness is estimated by interventions on the trace or the internal state, and its error includes the shift in distribution that interventions induce. Geometric coherence is estimated from hidden states with a manifold estimator, and its error includes the bias of the estimator and its dependence on the reference set. Correctness is established by checking the output against the task criterion, and for well-posed tasks it is essentially error free. Admissibility is established by evaluating the policy on the situation and the action, and its error is the error of the policy specification and of the representation of the situation.

Because the instruments differ, an agreement or disagreement between two properties is meaningful only after the measurement error of each has been considered. A weak association between validity and correctness, for instance, may reflect a validator with an overly strict convention, in which case valid traces are rarer than they should be and validity underpredicts correctness. The independence the monograph argues for is a logical independence of the properties and not a claim that the measurements are independent, which they may well not be.

2.10 Why the Properties Should Not Be Collapsed

A tempting simplification is to define a single quality score for a generation, combining the properties by weighted sum or conjunction. For some purposes this is reasonable. For the purpose of deciding what a system may be trusted to do, it is harmful, because the properties support different decisions. Correctness supports the decision to use the output as an answer. Validity supports the decision to read the trace as a record of a procedure. Faithfulness supports the decision to use the trace as a basis for diagnosing failures. Coherence supports the decision to treat the computation as operating in a familiar regime. Admissibility supports the decision to allow the action. A single score invites the use of one property in support of a decision that requires another, which is the structure of every error analyzed in the monograph. The six predicates are kept separate in the formal development so that each decision can be tied to the evidence it needs.

Chapter 3

The Epistemology of Generated Explanations

3.1 Five Kinds of Account

An explanation, a derivation, a justification, an execution trace, and an audit record are distinct kinds of account, and the vocabulary of machine reasoning tends to use them interchangeably. An explanation makes a result intelligible to a reader and may do so without being a record of how the result was reached. A derivation is a sequence of steps each licensed by a rule, and it establishes that a conclusion follows from premises whether or not any agent ever carried the steps out. A justification offers reasons for accepting or doing something and is addressed to an audience whose standards determine whether the reasons suffice. An execution trace records the state transitions of a process that actually ran. An audit record is an execution trace whose integrity and completeness are independently guaranteed. Only the last two bear, by their construction, on whether a process occurred, and only the last supports a claim about responsibility.

A text emitted by a generative model before its answer can have the form of any of these. What it is depends on how it was produced and on what independent evidence exists about that production. The failure analyzed in this chapter is the treatment of a text that has the form of an execution trace as though it had the evidential status of one.

3.2 Representation Does Not Establish Occurrence

The central epistemic principle can be stated compactly.

$$\text{Representation of a process} \not\Rightarrow \text{occurrence of that process.}$$

A linguistic representation of a computation is a string. That the string describes the steps of a search does not establish that a search took place, any more than a recipe establishes that a meal was cooked. The principle has a long history in the study of verbal reports about mental processes, where it was shown that people sometimes give fluent and confident accounts of the causes of their behavior that do not track the causes [18]. The point there and here is not that the reports are insincere. It is that production of a report and production of the reported process are different causal histories, and nothing in the form of a report

certifies that they coincide.

3.3 Declarations and Inspectable Evidence

A generated statement such as a claim to have checked the alternatives has the grammatical form of a first-person report. It differs from a report in the evidential sense because there is no independent record that particular alternatives were evaluated. The distinction between declaration and inspectable evidence is the distinction on which the rest of the monograph depends. Inspectable evidence is something that a party other than the producer can examine and that would differ if the claim were false: a recorded sequence of verified state transitions, a replayable log, an intervention whose outcome depends on the claimed intermediate content. A declaration is evidence only of what the producer is disposed to say.

3.4 Rationalization, Mechanism, Proof, and Operation

Four relations between an account and a result should be kept apart. Retrospective rationalization constructs an account after the result is fixed and is consistent with any result. Mechanistic explanation identifies the components and interactions that produced the result and is tested by intervention on those components. Mathematical proof establishes that a conclusion follows from premises and says nothing about whether any process arrived at the conclusion by that route. Operational evidence, such as a log produced by the executing system itself under controlled conditions, establishes that certain transitions occurred. A model's emitted trace may be any of these, and its being useful does not tell which.

3.5 Against Two Easy Responses

Two easy responses to the dissociation between trace validity and answer accuracy should be avoided. The first, anthropomorphic, treats the trace as a window onto thought and reads its apparent errors as human-like lapses. The second, deflationary, treats the trace as mere decoration and concludes that the model does not reason. Both substitute a stance toward the system for an investigation of it. The anthropomorphic reading predicts that interventions on the trace will change the answer in the way the trace says they should, and the deflationary reading predicts that they will not. Neither prediction is warranted in advance, and both can be tested. The position adopted here is that what a generated sequence warrants is a question for intervention and mechanistic measurement, not for interpretation of its surface.

3.6 Self-Report in People and in Models

The comparison with human verbal report is instructive in both directions. In people, reports about the causes of one's own behavior are often constructed by applying a plausible theory of why anyone would act that way, and they track the actual causes only when the actual causes happen to coincide with the plausible theory [18]. A language model trained on human-written explanations has access to the same plausible theories in large

quantity, and it can produce fluent explanations by applying them. The production of such an explanation is then a matter of the model's learned distribution over explanations, and its relation to whatever computation produced the answer is contingent. This is a point about the structure of the situation and not about the limitations of any particular model. It would hold for a model whose internal computation exactly matched its explanations only if the training had somehow selected for that match, and the training objectives in common use do not do so.

The comparison has limits. People have introspective access to some of their processes, and the literature on verbal report distinguishes cases in which reports are reliable from cases in which they are not. By analogy a model's reports may be reliable in some conditions, for example when the explanation is a simple restatement of an input feature that demonstrably drove the output. The monograph does not claim that generated explanations are unreliable in general. It claims that their reliability is a property to be measured by interventions and cannot be inferred from their fluency.

3.7 A Ladder of Evidence

Evidence that a generated trace describes the computation that produced the output can be arranged in a ladder of increasing strength. At the lowest rung is the declaration itself, the trace's own statement of what was done. Above it is consistency with the output, the observation that the trace leads to the answer given. This adds little, because a rationalization constructed after the answer is also consistent with it. The next rung is behavior under intervention on the visible text: if the trace is altered in a way that changes the content it claims, the output should change accordingly. Above that is behavior under intervention on internal representations, where the components that the trace claims to implement are manipulated and both the trace and the output are observed. At the top is an independent record produced by the executing mechanism, as distinct from a record produced by the generator, such as a log of verified state transitions maintained by the execution environment.

The ladder organizes the sections that follow. The experimental dissociation of Part III shows that the lower rungs do not reach the upper ones. The causal analysis of Part IV describes the middle rungs and their limits. The architecture of Part V places the top rung where it is needed, in the history written by the execution environment. The ladder also shows why anthropomorphic and deflationary readings are both premature, since each takes a position on a rung that has not been climbed.

3.8 What the Ladder Does Not Settle

The ladder concerns the relation between a trace and a computation. It does not settle whether the computation, whatever its relation to the trace, deserves to be called reasoning. That question depends on a stance about what reasoning is, and the monograph has no need to take one. A system can compute correct answers by a process that is not well described by any trace it emits, and it can do so while exhibiting the flexibility and generalization that make reasoning the natural description in other contexts. The point is that the description should be earned by evidence about the process, and the trace is not such evidence unless

the upper rungs of the ladder have been reached.

Chapter 4

Statistical Authority and Its Illegitimate Extensions

4.1 Descriptive and Normative Authority

Statistical authority is the evidential weight assigned to a system's predictions because of demonstrated performance. It is descriptive: it concerns how much a reasoner should update on what the system says. Normative authority concerns who or what is entitled to authorize an action. The two are related in practice, because evidence of reliability is often one of the considerations a policy takes into account when it grants authority, but they are not the same and the first never entails the second. A reliable forecaster has statistical authority about the weather and no authority to close an airport.

4.2 Sources of Inflated Authority

Several artifacts acquire authority beyond the evidence they provide. Benchmark scores support a claim about expected performance on a distribution resembling the benchmark and are routinely extended to claims about performance elsewhere. Confidence estimates support a claim about calibration under the conditions in which they were measured and are extended to claims about individual decisions. Successful tool calls support a claim that the calls were well formed and are extended to a claim that they were appropriate. Coherent explanations support a claim that the system can produce coherent explanations and are extended to a claim that the explanations describe the basis of its decisions. In each case the evidence is real and the extension is unsupported.

4.3 Authority Laundering

Authority laundering is a sequence of transformations through which a model-generated claim is converted into a permission without an independent authorization step. The transformations are individually unremarkable. A model summarizes a request. A downstream component reads the summary as an instruction. A tool executes the instruction. At no point did a party with the standing to grant permission grant it, yet the executed action carries the apparent endorsement of every stage that handled it. Chapter 22 gives the concept a formal treatment through provenance labels.

The concept is analytical. It describes a property of a pipeline, namely that authority can increase along it without a corresponding grant, and it does not attribute intent to any component. A pipeline can launder authority through nothing more than the ordinary convention that text produced by an upstream stage is treated as input of equal standing by the next.

4.4 The Parallel with Trace Validity

The two promotions identified in Chapter 1 can now be placed side by side. In the first, accuracy of the answer is promoted to validity of the trace. In the second, accuracy and fluency of the proposal are promoted to permission for the action. In both cases the promoted property belongs to a different evaluative domain from the evidence offered. The remedy in both cases is the same in structure: identify which evidence each property requires and refuse to let any one stand in for another.

4.5 A Typology of Promotions

Authority laundering and its relatives can be classified by the property that is promoted and the property it is promoted into. Promotion from accuracy to validity is the error of reading a correct answer as evidence that the stated procedure was followed. Promotion from validity to faithfulness is the error of reading a trace that conforms to a specification as a record of how the answer was reached. Promotion from reliability to permission is the error of reading a history of good outputs as a grant of standing to act. Promotion from confidence to decision is the error of reading a high stated confidence as a reason to dispense with review. Promotion from tool success to appropriateness is the error of reading the fact that a call returned without error as evidence that the call should have been made. Promotion from coherence to correctness is the error of reading a fluent and internally consistent account as evidence that it is accurate.

The list is not exhaustive, and its members share a structure: in each, a property that is observable and favorable is taken to establish one that is neither. The structure explains the persistence of the errors, since the evidence offered is real and is favorable, and the inference feels like a continuation of an argument that is otherwise sound. It also suggests a diagnostic that can be applied to any claim about a generative system: name the property the evidence establishes, name the property the conclusion asserts, and ask what links them. If the answer is a convention or a hope, the inference is a promotion.

4.6 Calibration and What It Licenses

A model is calibrated if, among the cases to which it assigns confidence q, a fraction q are correct. Calibration is a valuable property, and it is often cited as a ground for relying on a model's judgments. It licenses a specific inference: that the stated confidence is a reliable guide to the probability of correctness for cases like those on which calibration was measured. It does not license the inference that a decision to act should follow from high confidence, because the cost of error, the standing of the actor, and the availability of review enter the decision and are not captured by the probability of correctness. A model that is correct

with probability 0.99 and whose error would cause irreversible harm to a third party is not thereby authorized to act, and the figure 0.99 does not enter the question of authorization at all unless the policy makes it do so.

4.7 Authority Under Distribution Shift

Statistical authority is earned on a distribution, and it extends to another distribution only to the extent that the two resemble each other in the respects that matter. A model validated on historical requests has authority concerning requests of that kind. Deployment exposes it to requests of other kinds, including requests crafted by parties who know how the model behaves, and the authority earned in validation does not transfer to them. This is a limit on the descriptive notion, before the normative notion is considered. The practical consequence is that evidence of reliability should be reported with the distribution on which it was obtained, and that extensions of authority beyond that distribution should be recognized as extrapolations and handled with additional independent checks.

4.8 Self-Generated Evidence

A further source of inflated authority arises when the evidence for a system's judgment is produced by the system's own earlier choices [8]. In an interactive setting the history on which a justification is computed is not drawn independently of the policy. A choice that is repeated because earlier outcomes favored it generates further favorable outcomes in the cases it covers, and leaves unexplored the cases that could have contradicted it. Each observation is genuine, and the growth in the apparent justification reflects the policy's own sampling and not any change in the world. A system that treats its justification score as sufficient becomes most confident where its confidence has been least tested, and greater capacity to draw precise inferences from the sample sharpens the effect, since the inferences are precise about a sample the policy selected. This is the same structure as the promotion of accuracy to validity, in a setting where the history on which accuracy is assessed has itself been shaped by the system.

The point is related to the distinction between descriptive and normative authority. Evidence produced by a policy can raise statistical authority without any independent confirmation, and the consequence for permission is that a record of success accumulated under the system's own exercise of discretion is weaker support for extending that discretion than the same record accumulated under independent assignment.

Part II

The Geometric Substrate

Chapter 5

Manifold-Aligned Generative Inference

5.1 The Original Argument

The geometric account begins from the assumption that meaningful data occupy a lower-dimensional manifold $M \subset \mathbb{R}^n$ [5]. This is the manifold hypothesis in its usual form, for which there are both empirical motivation and a body of work on when it can be tested from samples [4]. At a regular point $x \in M$ the ambient space decomposes orthogonally into tangent and normal components,

$$\mathbb{R}^n = T_x M \oplus N_x M,$$

and any update direction $v(x)$ decomposes accordingly,

$$v(x) = \mathrm{Proj}_{T_x M}\, v(x) + \mathrm{Proj}_{N_x M}\, v(x).$$

The No-Noise Prediction Criterion of the original manuscript requires that a generative update carry no normal component,

$$\mathrm{Proj}_{N_x M}\, v(x) = 0. \tag{5.1}$$

The intended reading is that a generative system should move along directions supported by the structure of meaningful data and should not be trained to reproduce variation orthogonal to that structure.

A second element of the original treatment concerns generators. Let $G : \mathcal{Z} \to M$ be an immersion from a latent space. It induces the pullback metric

$$g_{ij}(z) = \Big\langle \frac{\partial G}{\partial z^i}, \frac{\partial G}{\partial z^j} \Big\rangle,$$

and inference can be formulated as a flow in latent coordinates, $\dot{z} = -\nabla_g S(z)$, where S is a semantic or variational potential and ∇_g is the gradient with respect to g. A point worth stating explicitly is that any velocity of the form $\dot{x} = dG_z(\dot{z})$ is tangent to the image of G by construction. Within a generator-parametrized dynamics the criterion (5.1) is therefore satisfied automatically with respect to the image of G, and the substantive question is

whether that image approximates the manifold of meaningful data.

5.2 What the Criterion Does and Does Not Say

The criterion needs three qualifications that the original formulation treated as background, and which the present monograph retains as part of the theory. The first is that the claim that noise lies in normal directions is an idealization of a particular data-and-noise model and not a property of noise in general. Isotropic Gaussian noise added in $\mathbb{R}^n$ to a point of a d-dimensional manifold has, in expectation, a fraction $(n-d)/n$ of its squared norm in the normal space, and a fraction d/n in the tangent space. Tangential noise therefore always exists, and it is negligible only when $d \ll n$. The second is that a learned manifold may be incorrectly specified, in which case the tangent space of the estimate is not the tangent space of the data. The third is that accurate prediction of clean data does not by itself prove tangent alignment of the underlying update, because an update can have a normal component that happens to cancel in the quantity being scored.

A useful connection to denoising makes the first qualification quantitative. If $x = x_0 + \sigma\varepsilon$ with Gaussian ε, the posterior mean of the clean point satisfies Tweedie's identity,

$$\mathbb{E}[x_0 \mid x] = x + \sigma^2 \nabla_x \log p_\sigma(x),$$

where p_σ is the density of the noised variable; the identity underlies the connection between score matching and denoising [27]. When the clean data concentrate near a manifold, the score $\nabla_x \log p_\sigma(x)$ points predominantly toward the manifold for small σ, that is, it has a dominant normal component, and the posterior mean is accordingly a tangentially close point. This is consistent with the intuition behind (5.1) but is a statement about a particular statistical model and not about every generative architecture.

5.3 From Generation to Computation

The step that motivates this monograph is the reading of the picture as a theory of computation and not only of sampling. The geometric account represents computation as constrained movement through a structured state space and does not require that the movement correspond to an intelligible sequence of explanations. A system that moves coherently through such a space can reach a state from which a correct output is likely without emitting, along the way, text that describes what moved it there. This is a hypothesis about autoregressive models and not a consequence of the manifold hypothesis, and it requires the representational work of Chapter 7 before it can be tested.

5.4 The Tangent-Normal Decomposition in Coordinates

The decomposition used in the criterion (5.1) has explicit forms that make clear what computing it requires. If the manifold is given by a parametrization $\phi : U \subset \mathbb{R}^d \to \mathbb{R}^n$ with Jacobian $J = D\phi(u)$ of full column rank at a point, the tangent space is the column space of J and the orthogonal projector onto it is

$$P_T = J(J^\top J)^{-1} J^\top, \qquad P_N = I - P_T.$$

If instead the manifold is given as a regular level set $\{F = 0\}$ of a smooth map $F : \mathbb{R}^n \to \mathbb{R}^{n-d}$ with Jacobian DF of full row rank, the normal space is the row space of DF and

$$P_N = DF^\top (DF\, DF^\top)^{-1} DF, \qquad P_T = I - P_N.$$

These are the standard formulas for embedded submanifolds [14]. They show that the condition $P_N v = 0$ is a system of $n - d$ linear conditions on the update, one for each independent normal direction, and that its force depends on how many directions are constrained. When d is close to n the condition is weak, and when $d \ll n$ it is strong. This observation matters later, because the dimension chosen for a local estimate of the manifold of hidden states directly controls how demanding the coherence criterion is.

5.5 Noise Models and the Conditions Under Which Normal Variation Dominates

The statement that noise lies in normal directions can be made precise under an explicit model. Let $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_n)$ and let P_N be a projector onto an $(n - d)$-dimensional normal space. Then $P_N \varepsilon$ is Gaussian in that subspace and

$$\mathbb{E}\|P_N \varepsilon\|^2 = \sigma^2 (n - d), \qquad \mathbb{E}\|P_T \varepsilon\|^2 = \sigma^2 d.$$

The normal component dominates in expectation by the factor $(n - d)/d$, and for image-like data, where d is plausibly much smaller than n, that factor is large. The tangential component is nevertheless nonzero, and it has a consequence that deserves emphasis: a model trained to remove the normal component of noise cannot in principle remove the tangential component, because the tangential component of the noisy point is itself a point of the manifold, indistinguishable from clean data without further information. What a denoiser can recover is the projection of the noisy point onto the manifold, and what it cannot recover is the tangential displacement of the clean point from its noisy position. The criterion should therefore be read as saying that the learnable part of the correction lies in the tangent directions of the clean-data prediction, and not as saying that all noise is orthogonal.

5.6 Prediction Targets and What They Imply

A diffusion-style model can be trained to predict the added noise, the clean data, or a mixture of the two, and the targets are related by an invertible linear map at each noise level. If $x = \alpha x_0 + \sigma \varepsilon$, then $x_0 = (x - \sigma \varepsilon)/\alpha$, so a model that predicts ε implicitly specifies x_0 and conversely. The equivalence is exact in the population limit and fails in practice because the errors of a finite model are distributed differently under the different parametrizations. When the target is the noise, the model's error is an error in an n-dimensional isotropic quantity, most of whose variance lies in the normal space, and any error in the normal component is mapped by the inverse relation into an error in the clean prediction scaled by σ/α. When the target is the clean data, the error is measured where the data lie. The original argument for predicting clean data is thus an argument about where the errors of an imperfect model are placed, and it is a claim of practical advantage and not of mathematical

necessity. That reading is the one adopted here, and it is consistent with the qualifications of the preceding section.

Chapter 6

Potentials, Stratification, and Descent

6.1 Gradient Flow and Descent

Let (M, g) be a Riemannian manifold and $S : M \to \mathbb{R}$ a smooth potential. The negative gradient flow $\dot{x} = -\nabla_g S(x)$ satisfies

$$\frac{dS}{dt} = \langle \nabla_g S, \dot{x} \rangle_g = -\|\nabla_g S\|_g^2 \leq 0.$$

The potential is therefore non-increasing along the flow and constant exactly at critical points. The inequality is the content of what is meant here by potential descent, which is weaker than Lyapunov stability of equilibria or convergence of trajectories, since those require additional assumptions. It is a statement about descent in the chosen potential, and it carries no information about whether the potential represents the task. A trajectory can converge to a stable critical point that corresponds to an incorrect answer, a spurious regularity learned from the training distribution, or an action that no policy permits.

6.2 Morse Structure

When S is a Morse function, its critical points are non-degenerate, and the topology of sublevel sets changes only at critical values, in the manner described by Morse theory [17]. The earlier work uses this structure to describe cognitive updates as descents between basins associated with critical points of different index, with the loops of the CLIO formulation corresponding to cycles of descent and perturbation. CLIO denotes the cognitive-loop formulation of the earlier manifold-aligned inference manuscripts [6], and only its reading as alternating descent and perturbation is used here. The monograph uses this machinery only in the following restricted sense. A trajectory can be described by the sequence of basins it traverses, and two trajectories can be compared by whether they enter the same final basin. Which basins correspond to correct outputs is determined by the task, not by the geometry.

6.3 Stratified State Spaces

A single smooth manifold is a poor model for representations that mix discrete and continuous structure. Stratified spaces, in which the state space is a union of manifolds of different dimension glued along lower-dimensional boundaries, accommodate such mixtures. On a stratified space the notion of tangent direction becomes dependent on the stratum, and updates that cross a stratum boundary change the dimension of the space of admissible directions. This matters for the argument because the boundaries of strata are natural candidates for the places where the behavior of a system changes qualitatively, and because the corresponding boundaries of a feasible region in the admissibility theory of Part V have the same formal character.

6.4 Three Conditions Kept Apart

Three conditions emerge and are kept apart throughout. Geometric coherence is the condition $\dot{x} \in T_x M$. Potential descent is the condition $dS/dt \leq 0$. Execution admissibility is the condition $A_\Pi(x, a) = 1$. The first two are properties of a dynamical system relative to a manifold and a potential. The third is a property of an action relative to a policy. A system can satisfy the first two and fail the third, because tangency and descent say nothing about which regions of the manifold the policy excludes.

6.5 Gradient Flow in Latent Coordinates

When the dynamics are expressed in latent coordinates of a generator G, the flow takes a concrete form. Let $S : \mathcal{Z} \to \mathbb{R}$ be a potential on latent space and let $g(z) = DG(z)^\top DG(z)$ be the pullback metric, assumed positive definite where G is an immersion. The Riemannian gradient is $\nabla_g S = g^{-1} \nabla_z S$ with $\nabla_z S$ the Euclidean gradient, and the flow is

$$\dot{z} = -g(z)^{-1} \nabla_z S(z).$$

The image $x(t) = G(z(t))$ has velocity $\dot{x} = DG(z)\dot{z}$, which lies in the column space of $DG(z)$, namely the tangent space of the image of G. Tangency is therefore a structural consequence of the parametrization, as noted earlier. The descent property follows from

$$\frac{dS}{dt} = \nabla_z S^\top \dot{z} = -\nabla_z S^\top g^{-1} \nabla_z S \leq 0,$$

with equality only where $\nabla_z S = 0$ because g^{-1} is positive definite. The flow is invariant under reparametrization of the latent space, in the sense that if $z = \psi(w)$ for a diffeomorphism ψ, the pushed-forward flow coincides with the gradient flow of $S \circ \psi$ under the metric pulled back through ψ. This invariance is the reason for using the Riemannian gradient and not the Euclidean one, which depends on the arbitrary choice of coordinates.

6.6 Gradient Flows Do Not Cycle

A consequence of the descent property bears on how the cognitive loops of the earlier formulation can be understood. Suppose a trajectory of a gradient flow is periodic with

period $T > 0$, that is $z(T) = z(0)$. Then $S(z(T)) = S(z(0))$. But S is non-increasing along the flow and strictly decreasing at any time at which the gradient is non-zero. If the gradient is non-zero at some time on the orbit, the strict decrease over an interval gives $S(z(T)) < S(z(0))$, a contradiction. The gradient must therefore vanish along the whole orbit, which makes the orbit a single critical point. A pure gradient flow has no non-trivial periodic orbits.

Loops that represent repeated cycles of an inferential process therefore require something beyond a gradient flow with a fixed potential. The candidates are a time-dependent potential, as when attention shifts between objectives, stochastic perturbation that kicks the state out of one basin and allows the flow to descend into another, discrete resets that restore a state from memory, or non-gradient terms in the vector field such as rotational components. This is worth stating because it constrains the reading of the earlier cognitive loops: they are not trajectories of a descent on a single fixed potential, and a theory in which they are must specify the additional mechanism that closes the loop.

6.7 Discrete Updates and Retractions

The flows above are continuous, whereas the computation of a transformer proceeds in discrete steps, one per layer or per token. A discrete counterpart takes a step of size η in the direction of the negative Riemannian gradient and returns to the manifold by a retraction R_x, a smooth map from the tangent space to the manifold that agrees with the identity to first order at the origin [1]. The update is $x_{t+1} = R_{x_t}(-\eta \nabla_g S(x_t))$. For a sufficiently small step size and a retraction of second order, the potential decreases at each step under standard smoothness conditions, with a decrease of the order of $\eta \|\nabla_g S\|^2$ for small η. When the step size is large relative to the curvature, descent is not guaranteed and the discrete trajectory can depart from the manifold or oscillate. For the analysis of hidden states this matters because the steps between successive token positions are not small in any obvious sense, and the discrete trajectory should not be assumed to track a continuous flow without a check on the ratio of step length to curvature scale.

Chapter 7

Hidden States as Approximate Manifolds

7.1 The Representation Problem

Applying the geometric account to a transformer requires choices that the differential geometry does not supply. Hidden states of an autoregressive transformer are vectors in a high-dimensional Euclidean space, produced layer by layer, at each token position. They do not automatically inhabit a smooth semantic manifold. Discrete token transitions do not automatically correspond to tangent flows. Any claim that the model's computation moves along a manifold is therefore a claim about an approximation, and its content depends on the representation chosen, the estimator used to approximate the manifold, and the tolerance with which departures are measured.

7.2 Choosing a Representation

Let $h_t^{(\ell)} \in \mathbb{R}^D$ denote the residual-stream state at position t and layer ℓ. A representation is a choice of layer or layers, a choice of position, and possibly a linear projection to a lower-dimensional subspace. Different choices can give different conclusions, and results should be reported for a stated family of choices and not for a single selected one. The reference set from which a local manifold is estimated is also a choice. It should be drawn from states reached on a held-out set of correct completions of the same task family, so that the manifold estimates task-relevant states and not all states the model can occupy.

7.3 Local Estimators and Off-Manifold Residuals

Given a reference set $\mathcal{R}$ of hidden states and a state h, define the local tangent estimate by principal component analysis of the k nearest neighbors of h in $\mathcal{R}$, retaining d components. Let P_h denote the orthogonal projector onto the span of those components, centered at the neighbor mean μ_h. The off-manifold residual is

$$r(h) = \frac{\|(I - P_h)(h - \mu_h)\|}{\|h - \mu_h\|}.$$

For a trajectory $h_0, h_1, \ldots, h_k$ the corresponding step residual uses the secant $\Delta_t = h_{t+1} - h_t$,

$$\rho_t = \frac{\|(I - P_{h_t})\Delta_t\|}{\|\Delta_t\|},$$

and the trajectory residual is the mean of ρ_t over steps. A small trajectory residual means that the sequence of internal updates is nearly tangent to the estimated manifold. The secant is a discrete surrogate for the velocity and the surrogate error depends on the step size relative to the curvature of the manifold, so the residual should be accompanied by a null comparison, namely the same statistic computed for random directions of equal norm and for trajectories on incorrect completions.

Definition 7.1 (Estimated geometric coherence)**.** Fix a representation, a reference set $\mathcal{R}$, neighbor count k, dimension d, and tolerance ϵ. A trajectory is $(\mathcal{R}, k, d, \epsilon)$-coherent if its trajectory residual is at most ϵ. The predicate $\Gamma(x, z)$ is the indicator of coherence for the trajectory induced by z on task x.

The definition makes explicit that coherence is relative to the estimator. A conclusion that depends on a particular k or d is a conclusion about that estimator. Claims intended to bear on the model rather than on the estimator should hold across a range of reasonable settings, and the experimental protocol of Chapter 27 specifies such a range.

7.4 Relating Token Interventions to State Transitions

The remaining gap is the connection between what is done to tokens and what happens to states. A prefix replacement changes the input to every later computation and so changes the entire downstream trajectory. The monograph treats the relation between the token-level intervention and the induced change in trajectory residual as an empirical quantity to be measured, and it does not assume that a change in the surface form of a trace corresponds to a particular direction in state space. Where a measured relation is found, it supports the geometric account. Where it is not found, the account is not supported for that model and task.

7.5 Curvature and the Bias of Secant Estimates

The trajectory residual uses secants between successive states, and secants are biased estimates of tangent directions when the underlying curve bends. For a smooth curve of curvature κ and a chord of length ℓ, the angle between the chord and the tangent at either endpoint is approximately $\kappa\ell/2$ for small $\ell\kappa$, and it is approximately zero at the midpoint. The normal fraction of the chord relative to the tangent at an endpoint is therefore of order $\kappa\ell/2$, a quantity that grows with step length and curvature even for a trajectory lying exactly on the manifold. A trajectory residual of a given size is thus compatible with perfect alignment if the steps are long relative to the radius of curvature. This is the reason the definition of coherence in Chapter 7 requires a comparison with null trajectories, and why the residual should be compared at matched step lengths. A residual measured on a trajectory with long steps cannot be compared with one measured on a trajectory with short steps without correcting for this term.

A related bias affects the local estimate of the tangent space. Principal component analysis on the k nearest neighbors recovers the tangent space well when the neighborhood is small relative to the curvature and large enough to average out noise. Too small a neighborhood gives a noisy estimate, and too large a neighborhood includes points from regions where the manifold has curved away, inflating the apparent variance in normal directions. The choice of k trades variance against bias, and no value is correct in general. The prescription is to report results over a grid of k and to treat conclusions that hold only at an isolated value as unreliable.

7.6 Choosing the Dimension and the Layer

The dimension d of the local estimate can be chosen by the fraction of variance explained, by a gap in the spectrum of the local covariance, or by a fixed value used across all states. Each choice has a defect. A variance threshold gives different dimensions at different points, which complicates comparison. A spectral gap may be absent. A fixed value ignores genuine variation in local dimension. The recommended practice is to compute the residual at several values of d and examine how it varies, because a residual that falls steeply as d rises toward the intrinsic dimension and then flattens indicates a well-defined manifold, and one that falls gradually indicates that the states fill the space and no useful low-dimensional structure has been found.

The layer matters for similar reasons. Early layers hold representations close to the token embeddings, and late layers hold representations close to the output logits, and the geometry in each reflects different computations. The middle layers often carry the most abstract task-relevant information. The monograph does not claim a preferred layer. The test of Hypothesis 16.2 should be run across layers, and a result should be considered robust only if the improvement in prediction appears over a contiguous range of layers and not at one.

Part III

Reasonless Intermediate Tokens

Chapter 8

Formal Reconstruction of the Source Experiments

8.1 The Domain

The source experiments [26] concern maze solving. A task x is a maze with a start and a goal, serialized as a token sequence. The reference algorithm is A*, which maintains a priority queue of frontier cells ordered by the sum of path cost so far and a heuristic estimate of remaining cost, and which repeatedly expands the cell of least priority. Execution of A* on a maze induces a deterministic sequence of operations, namely the expansion of a cell and the insertion of its neighbors with their computed costs. A trace serialization writes this sequence as text. A formal validator $V_{\mathcal{S}}$ parses a candidate trace and checks, operation by operation, that it is the sequence the algorithm would perform on the given maze, and that the final answer is the path recovered from the final state. The specification $\mathcal{S}$ is thus the execution semantics of A* together with the serialization grammar.

8.2 Three Training Regimes

Three training regimes are compared. In the solution-only regime the model is trained to emit the final path directly. In the correct-trace regime it is trained to emit the A* trace of the given maze followed by the path. In the swapped-trace regime each training example pairs a maze with the A* trace of a different maze and the correct path for the original maze, so that every trace is a syntactically well formed A* trace that is nonetheless not the trace of the maze it accompanies. The swapped regime is the experimental device that separates the form of a trace from its semantic relation to the task: it preserves the grammar, the vocabulary, the typical length, and the marginal statistics of traces while destroying their correspondence with the problem.

Evaluation is carried out on several maze distributions. Cross-distribution evaluation is informative because a model that has learned the procedure would be expected to transfer across distributions in which the procedure is the same, while a model that has learned distribution-specific regularities would be expected to degrade. In the source study the training mazes are generated by Wilson's algorithm in one setting and by a cave-style generator (SF-style) in another, and evaluation covers several generators including Kruskal, depth-first search, the Drunkard's Walk, and SF-style mazes, with differences in transfer

reported between the regimes.

8.3 Reported Findings

The findings relevant to this monograph, stated as findings of the source study, are three. First, models trained on swapped traces reach high final-answer accuracy, and on some evaluation distributions exceed models trained on correct traces, in particular for the SF-style models on Drunkard and SF-style mazes. Second, trace validity as measured by the formal validator and answer accuracy diverge: models can produce correct answers with invalid traces. Third, in some post-training experiments with group relative policy optimization, answer accuracy improved while trace validity deteriorated. A further finding concerning partially swapped datasets is discussed in Chapter 11.

8.4 The Precise Conclusion and Its Limits

The conclusion supported is the following.

> In the evaluated settings, high final-answer accuracy does not require the model to generate a trace that is valid under the prescribed A* execution semantics.

The conclusion is a negative existence claim about a necessary condition, and its strength should not be inflated. It does not follow that the models performed no meaningful internal computation, because the answers were correct and something computed them. It does not follow that intermediate tokens never carry procedural content in other models or tasks. It does not follow that the traces had no causal influence on the answers, which is an interventional question the training-regime comparison does not address. What follows is that the semantic validity of the trace is not what accounts for the accuracy, and the explanatory burden shifts to whatever else does.

8.5 What the Validator Checks

The formal validator is the instrument on which the central finding rests, and its construction determines what the finding means. A validator for A* traces must establish several things for each operation recorded in the trace: that the cell expanded is the one with least priority in the queue as it stands after the preceding operations, that the neighbors inserted are the neighbors of that cell that are not walls, that the costs assigned to them are the correct sums of path cost and heuristic, and that ties are broken in the manner the specification prescribes. The last requirement is a source of subtlety, because A* as an algorithm does not fix a tie-breaking rule, and a validator must choose one. A trace that is a correct execution of A* under one tie-breaking rule may be rejected by a validator that assumes another. The reported validity rates are therefore rates of conformity to a particular serialization and tie-breaking convention, and an analysis that reads them as rates of algorithmic correctness in a broader sense should state the convention.

A second subtlety concerns what the validator does with a trace that is correct up to a point and then deviates. A binary validator rejects the whole trace. A finer validator can report the position of the first deviation, the fraction of operations that are correct,

or the longest correct prefix. These measures are not interchangeable, and a model that produces mostly correct traces with occasional errors differs from one that produces traces that are wrong from the outset. The binary measure is the most conservative and the least informative, and the graded measures are preferred where the object of study is how the traces degrade.

8.6 The Swapped Regime as an Intervention on Supervision

The swapped regime has the logical structure of a controlled intervention on the training data. It holds fixed the marginal distribution of traces, because the set of traces used is the same as in the correct regime, and holds fixed the marginal distribution of mazes and of answers. It breaks the dependence between the maze and its trace, replacing it with independence. The dependence between the maze and the answer is untouched. The comparison between the correct and swapped regimes therefore isolates the contribution of the trace-maze dependence to the performance of the trained model, with everything else held constant, and a performance difference between the regimes is attributable to that dependence under the usual assumptions of a controlled experiment.

The contrast with the solution-only regime has a different logical structure. The solution-only model has no trace at all, so its comparison with the swapped model isolates the effect of having additional token positions with a particular marginal distribution of content. If the swapped model outperforms the solution-only model, the additional positions are useful even when their content is independent of the task during training. This is the comparison most relevant to the scaffolding hypothesis. If the swapped model does no better than the solution-only model, the utility of the correct trace would have to lie in its dependence on the maze, which the swapped regime has removed.

8.7 Reading Cross-Distribution Results

Evaluation on distributions other than the training distribution tests whether what the model learned is specific to the training mazes. A model that has learned the procedure should transfer, to the extent that the procedure is the same on the new mazes and the model can represent the new inputs. A model that has learned heuristics adapted to the training distribution, such as typical corridor shapes, should degrade. The inference from transfer to procedure is weaker than it appears, however, because transfer can also occur when the new distribution shares statistical structure with the old one, and the degree of shared structure is not controlled by the experimenter unless the distributions are constructed to vary it. The mazes described in the source as Drunkard-style and SF-style differ in their generation processes, and the extent to which they differ in the features relevant to a learned heuristic is an empirical matter that the performance numbers alone do not settle.

8.8 Threats to Inference

Four threats limit what can be concluded from the experimental design. The first is scale: the models are small and trained on a narrow task, and results for such models need not hold for large pretrained models whose representations are shaped by far broader data. The

second is the choice of task: maze solving by A* is a deterministic algorithmic problem with a clean specification, and the dissociation observed there may not carry over to tasks with weaker specifications, such as open-ended mathematics or planning in an uncertain world. The third is the choice of specification: validity is conformity to one serialization of one algorithm, and other characterizations of what the model does might classify the same traces differently. The fourth is the training procedure: the results were obtained under particular optimizers, learning rates, and data volumes, and the dependence on these is not known from a single study. None of the threats undermines the narrow conclusion stated above, which is a statement about the evaluated settings. They bear on how far the conclusion may be extended, and the monograph extends it only as a hypothesis for further testing.

Chapter 9

Information, Computation, and Intermediate Tokens

9.1 Problem Information and Computational Transformation

A distinction between information about the problem and transformation of that information is needed to state what intermediate tokens can and cannot contribute. Let x denote the task, z the intermediate sequence, and y the final answer. An autoregressive model factors the joint distribution as

$$p_\theta(z, y \mid x) = p_\theta(z \mid x)\, p_\theta(y \mid x, z).$$

Let $y^* = f(x)$ denote the answer defined by the task. Because z is generated from x by the model, possibly with independent sampling noise, the variables form the Markov chain $y^* \to x \to z$, and the data processing inequality gives $I(y^*; Z) \leq I(y^*; X)$. The sequence z therefore cannot contain more information about the correct answer than x already contains. The inequality does not say that z cannot help. It says that any benefit from z is a benefit of computation and not of new external information: z can make information already present in x available in a form that the next-token computation can use within its fixed depth.

9.2 Training Traces and Generated Traces

A distinction between two sources of z matters for interpretation. In training, a trace is a supervised target. In the swapped regime it is statistically independent of the specific maze it is paired with, given the maze distribution. At inference, the trace is generated by the model from the input and is therefore statistically dependent on the input even when it is procedurally invalid. A generated invalid trace can encode functions of the input. This is the reason that the swapped-regime finding does not imply the traces are uninformative at inference: the training supervision was uninformative, but the learned generator is not required to reproduce that uninformativeness.

9.3 Extra Tokens as Computational Opportunity

A transformer of fixed depth performs a bounded amount of serial computation per token. Additional tokens provide additional passes, with the intermediate results held in the sequence

that later positions attend to. In this respect intermediate tokens resemble scratchpads [19], iterative algorithms that reuse a state, recurrent computation, and learned latent workspaces. The resemblance is structural and should not be taken to imply equivalence. A scratchpad whose contents are semantically meaningful differs from one whose contents are an uninterpreted code, even if both give the network additional serial steps. The comparison that matters is between sequences matched on length and token budget and differing in semantic relation to the task, which is the comparison the swapped regime provides.

9.4 The Serial Depth Argument

A transformer with L layers performs at most L sequential applications of its layer stack to the information at any given position, and the information available to a position is limited to what earlier positions have computed and exposed through attention. When the model produces a token and that token is fed back as input, the next position can use the result of the preceding computation as though it were part of the input, and the effective serial depth of the whole computation increases by up to L. For a sequence of k generated tokens the serial depth available is of the order of kL, instead of L. Problems whose solution requires serial composition of more than L dependent operations cannot be solved in a single forward pass, under standard assumptions about what a bounded-depth network computes, and can be solved with a sufficient number of intermediate tokens that carry the intermediate results.

The argument establishes that additional tokens are useful for a reason that has nothing to do with their semantic content, namely that they provide additional serial steps. It also shows what is required of them: to carry intermediate results forward, the tokens must encode those results in a way that later positions can decode. This is a requirement on the code, which need not be the code of the prescribed algorithm. A model that has learned its own code for intermediate results can use tokens as a scratchpad whose entries are meaningful to itself and not to a reader who applies the A* specification. The swapped-trace result is consistent with this reading, in which the learned code diverges from the prescribed one, and it is also consistent with readings in which the tokens carry no intermediate results at all. The serial depth argument by itself does not distinguish these.

9.5 Exposure and Self-Conditioning

During training with supervised traces, the model is conditioned on the correct previous tokens at every position, a procedure known as teacher forcing. At inference, the model is conditioned on its own previous outputs. If its outputs contain errors, later positions see inputs unlike any they were trained on, and the effect of an error can compound. This mismatch between training and inference conditions has two relevant consequences. First, a model trained on correct traces may produce traces whose later portions are degraded by earlier errors, and such traces will fail a strict validator while the answer, if it depends on a computation less sensitive to the errors, remains correct. Second, a model trained on swapped traces never sees traces that correspond to its inputs, and so cannot have learned to rely on them for the content of the answer. It must learn to compute the answer from the input by some route that does not require the trace to be a reliable record. Whether

it then uses the trace at all is an empirical question that the interventional comparison of Chapter 10 addresses.

9.6 Entropy of Generated Traces

The statistical dependence of generated traces on the input can be quantified. For a trained model, the conditional entropy $H(Z \mid X)$ of the generated trace given the task measures how much of the trace is determined by the task, and the mutual information $I(Z;X) = H(Z) - H(Z \mid X)$ measures how much information about the task the trace carries. A model that has learned a procedure produces a trace with high mutual information with the task, because the trace is a deterministic function of it. A model that emits a task-independent filler produces a trace with mutual information near zero. Generated traces of models trained on swapped data are an empirical object whose mutual information with the task is not known in advance, and measuring it distinguishes the case in which the model has learned to produce task-dependent invalid traces from the case in which it produces task-independent text. The first would suggest a learned code, and the second a pure serial-computation affordance. This measurement can be carried out with standard estimators on the generated traces and does not require access to the model's internal states.

Chapter 10

The Computational Scaffolding Hypothesis

10.1 Statement

Hypothesis 10.1 (Computational scaffolding)**.** Intermediate tokens can function as computational scaffolds whose utility is not reducible to their apparent procedural semantics. The utility depends on properties of the sequence, such as its length, its statistical regularity, and the conditional structure the model has learned to associate with it, that are partly or wholly independent of whether the sequence is a valid derivation.

The hypothesis is deliberately weaker than the geometric hypothesis of Part IV. It asserts only that semantic validity is not the sole source of utility, and it is compatible with several mechanisms, including provision of additional serial computation, stabilization of an internal state through repeated conditioning, and priming of a mode of processing.

10.2 An Interventional Measure of Utility

Utility is defined by intervention. Let z_0 be a specified control sequence and let $\mathrm{do}(Z = z)$ denote controlled replacement of the generated prefix by z. The utility of z on task x relative to z_0 is

$$\Delta U(z; x) = \mathbb{P}\big(C = 1 \mid x, \mathrm{do}(Z = z)\big) - \mathbb{P}\big(C = 1 \mid x, \mathrm{do}(Z = z_0)\big).$$

This quantity depends on the intervention protocol and on the control, and it is not an intrinsic measure of the semantic value of z. Two different controls, such as an empty prefix and a prefix of random tokens of equal length, answer different questions. The first asks whether any prefix helps. The second asks whether the specific content of the prefix helps beyond occupying positions.

10.3 What Scaffolds Might Depend On

The hypothesis suggests a structured investigation of what a useful scaffold depends on. Candidates are length, syntax, domain-specific vocabulary, statistical regularity of the token sequence, and learned conditional structure, meaning the dependence of later tokens on earlier ones as the model has learned it. Each can be varied while the others are held fixed.

Repeated tokens preserve length and vocabulary while removing structure. Random tokens preserve length while removing vocabulary and structure. Shuffled valid traces preserve vocabulary and marginal structure while removing the dependence on the task. Learned prefixes, optimized by gradient descent against task success with no constraint on form, preserve nothing but the optimization target and indicate how much utility is available at all.

10.4 Proposed Comparisons

The comparison suite is a factorial design of prefix type against matched computational budget. The prefix types are correct traces, swapped traces, random tokens, repeated tokens, and learned prefixes, each evaluated at token counts matched to those of the correct traces. The outcome is answer accuracy on held-out mazes from each evaluation distribution, and the analysis estimates ΔU for each type relative to the empty prefix and relative to the random prefix. A finding that swapped traces and random tokens give equal utility at equal length would favor a pure serial-computation account. A finding that swapped traces outperform random tokens would indicate that the learned conditional structure contributes. The design is specified in full in Appendix D.

10.5 A Taxonomy of Mechanisms

Several mechanisms can account for a case in which a trace is useful and invalid, and they differ in what they predict. A serial-computation mechanism holds that the tokens supply extra forward passes, and that their content is incidental provided the length is sufficient. It predicts that random tokens of the same length are as useful as swapped traces and that repeated tokens are nearly so. A learned-code mechanism holds that the model has developed its own encoding of intermediate results in the token stream, so that the tokens carry information used by later positions. It predicts that swapped traces are more useful than random tokens, that the tokens have high mutual information with the task, and that replacing a generated trace with another trace from the same model on a different maze lowers accuracy. A priming mechanism holds that the tokens place the model in a mode of processing, much as a task description does, and that their specific content matters little beyond identifying the mode. It predicts that a short prefix of the right type suffices and that length beyond some threshold adds nothing. A distribution-matching mechanism holds that the trace keeps the model's inputs within the range of its training distribution, so that the later computation operates in a familiar regime. It predicts that traces from the model's own distribution are useful and that out-of-distribution traces, even if valid, are harmful.

These predictions are distinguishable by the factorial design of the preceding section, supplemented by measurements of mutual information and by cross-maze replacement. No single mechanism is expected to account for everything, and the mixture of mechanisms may differ across models and tasks. The relevance for the geometric account is that each of them is compatible with a description in terms of trajectories: serial computation lengthens the path, a learned code shapes it, priming sets its starting region, and distribution matching keeps it near familiar states. The geometric account is a way of describing the effects and does not by itself choose among the mechanisms.

10.6 Relation to Soft Prompts and Learned Prefixes

The learned-prefix arm of the design has a close relative in the work on prompt tuning, in which continuous vectors prepended to the input are optimized by gradient descent to steer a frozen model toward a task [15]. Such prefixes have no semantic interpretation as text, and yet they can be as effective as full fine-tuning in some regimes. This is a demonstration that the utility of a prefix need not depend on its being interpretable as language. The difference from the intermediate tokens studied here is that soft prompts are continuous and are optimized with the model frozen, whereas intermediate tokens are discrete and the model is trained to produce them. The existence of effective uninterpretable prefixes supports the plausibility of the scaffolding hypothesis and does not establish it for generated discrete tokens.

10.7 Statistical Considerations

The comparisons proposed involve differences in accuracy that may be small, evaluated on models trained with different random seeds, and the analysis must respect that structure. Variation across seeds can be as large as the differences between conditions, so conclusions require several seeds per condition and an analysis that treats the seed as the unit of replication for claims about training, and the maze as the unit for claims about evaluation. The comparisons among five prefix types and several evaluation distributions involve many hypothesis tests, and a correction for multiple comparisons or a hierarchical model should be used. Effect sizes should be reported with intervals, and a failure to detect a difference should be reported with the interval that shows how large a difference the experiment could have detected, since a null result from an underpowered comparison does not support equivalence.

Chapter 11

The Generalization Paradox

11.1 The Observation

A surprising feature of the source results is that training on correct algorithmic traces can generalize less well to some distributions than training on mismatched ones. The observation is surprising only against the background expectation that supervision closer to a correct algorithm should yield behavior closer to it. The source study also reports a U-shaped performance trend across partially swapped datasets, in which accuracy is not monotone in the fraction of swapped traces. The shape is reported as an observation, and this chapter treats its explanation as open.

11.2 Competing Explanations

Four families of explanation can be distinguished, and they make different predictions. Overconstraint holds that correct traces impose a specific serialization of the search that the model must reproduce exactly, and that errors in reproduction propagate, whereas a model free of such constraint can compute the answer by whatever route it finds. Imitation of incidental details holds that correct traces contain regularities particular to the training distribution, such as typical expansion orders in a given maze style, that the model learns and that fail to transfer. Conflicting supervision holds that when the trace and the answer are produced by the same network under a single loss, the demands of faithfully reproducing the trace and of computing the answer pull in different directions. Optimization dynamics holds that the differences are properties of the loss landscape reached from the different data and not of any representational difference. The families are not exclusive.

11.3 Competence Versus Task Completion

Genuine algorithmic generalization should be distinguished from successful task completion on unfamiliar input distributions. A model that solves unfamiliar mazes has generalized in the weak sense that its answers are correct. It has generalized in the strong sense only if what it does on the new distribution is the same computation that it does on the old. The two senses coincide for a model that implements the algorithm and come apart for a model that implements a collection of distribution-specific heuristics that happen to succeed. Evidence for the strong sense requires examining the internal computation, and

the geometric measurements of Chapter 7 are one candidate.

11.4 An Experimental Design

To separate algorithm-specific imitation from broader computational competence, train models in each regime on a family of maze distributions that differ along controlled axes, evaluate on held-out distributions that vary along each axis separately, and measure both answer accuracy and the agreement of internal trajectories across distributions, for instance by the similarity of the sequences of basins traversed. If the correct-trace regime shows lower answer transfer but higher internal agreement, algorithm-specific imitation without competence on novel structure is indicated. If the swapped regime shows higher answer transfer with lower internal agreement, success by distribution-specific routes is indicated.

11.5 A Stylized Model of the Non-Monotone Pattern

A simple illustrative model shows how a non-monotone dependence on the fraction of swapped traces can arise without any exotic mechanism. The model is offered only to show that the pattern is not paradoxical, and it makes no claim about the source experiments. Suppose a model can reach an answer by two routes. The first route uses the trace as a record of intermediate results and is learned to the extent that the training traces are correct, so its strength grows with the fraction of correct traces. The second route computes the answer without relying on the trace and is learned to the extent that the trace is unreliable, so its strength grows with the fraction of swapped traces. At a fraction near zero swapped, the first route is strong and the second weak, and the model performs well on the training distribution through the first route. At a fraction near one, the second route is strong and the first weak, and the model performs well through the second. At intermediate fractions, neither route is fully developed, because the supervision for each is contaminated by the other, and the model may be left with two weak routes and a conflicted treatment of the trace. Performance may then dip, producing a U-shaped dependence.

The model makes testable predictions. It predicts that at intermediate fractions the answer depends on the trace to an intermediate degree, so that the interventional influence of the trace varies monotonically with the fraction even though accuracy does not. It predicts that the internal trajectories at intermediate fractions show a mixture of the signatures of the two routes. These predictions can be checked against the interventional and geometric measurements proposed in the monograph, and they distinguish the stylized model from alternatives in which the dip arises from optimization difficulty without any change in routes.

11.6 What Each Explanation Predicts

The four families of explanation separate under suitable measurement. Overconstraint predicts that models trained on correct traces fail primarily by compounding errors in the trace, so that failures cluster in long traces and in the later portions of traces, and that giving the model the correct prefix at inference restores its accuracy. Imitation of incidental details predicts that models trained on correct traces show distribution-specific regularities in their traces, such as characteristic expansion orders, which transfer poorly, and that

accuracy on held-out distributions correlates with the degree to which the trace departs from the training distribution's regularities. Conflicting supervision predicts that the loss on the trace and the loss on the answer trade off during training, which can be seen in the training curves, and that weighting the answer loss more heavily improves transfer. Optimization dynamics predicts that the differences between regimes diminish when the optimizer and schedule are tuned separately for each regime, and persist only under a fixed shared configuration. These are cheap tests relative to the experiments of the program, and they should be run first, since a positive result for any of them changes the interpretation of the swapped-trace advantage.

Chapter 12

Trace Length and the Appearance of Adaptive Deliberation

12.1 The Comparison

The source study compares the lengths of generated traces with the lengths of reference A* traces on the same mazes. A positive relation between generated length and a measure of problem difficulty is often read as evidence that the model allocates more deliberation to harder problems. The reading requires care at each step.

12.2 Measures of Association

The Pearson correlation measures linear association and is sensitive to outliers. Rank correlation measures monotone association and is robust to nonlinear but monotone relations. Neither detects non-monotone relations, and both are distorted by censoring when generated traces are truncated at the context limit, because the longest traces are then recorded at the limit regardless of what they would have been. An analysis of length should report the association by more than one measure, state the censoring, and examine the relation graphically.

12.3 Difficulty Is Relative to an Algorithm

The number of expansions that A* performs on a maze is an algorithm-relative measure of difficulty. A different algorithm, or the same algorithm with a different heuristic, yields a different count on the same maze. It is not an intrinsic measure of computational complexity and should not be treated as one. A model whose trace length tracks A* expansion count is tracking a property of the reference algorithm, which could reflect either the use of something like that algorithm or the correlation of both quantities with some third property of the maze, such as its size or branching.

12.4 Four Distinct Quantities

Four quantities should be defined separately. Task difficulty is a property of the task relative to a stated solver. Computational expenditure is the amount of computation actually performed by the model, which is not directly observed. Generated sequence length is the observed count of tokens. Problem-adaptive resource allocation is a relation between difficulty and expenditure, namely that expenditure increases with difficulty beyond what is explained by input size. Sequence length is an observable proxy for expenditure, and expenditure is an unobserved quantity that the proxy approximates only to the extent that tokens correspond to units of computation.

12.5 Tests for Adaptive Allocation

Tests for whether a model allocates computation according to uncertainty, task structure, or distributional familiarity can be built from controlled variation. Hold input size fixed and vary solution structure to test the dependence on task structure. Present tasks from a familiar distribution and matched tasks from an unfamiliar one to test familiarity. Measure the model's own uncertainty over the answer at the point where the trace begins, and test whether trace length is predicted by that uncertainty after controlling for size and structure. Each test isolates one of the three candidate bases of allocation and a model may show more than one.

12.6 Censoring Formalized

Let L^* be the length the model would generate without a limit and let c be the context limit. The observed length is $L = \min(L^*, c)$. For tasks whose natural length exceeds c, the observed length is c regardless of how much longer the trace would have been, and the observed data have a point mass at c. A correlation between L and a difficulty measure computed on such data underestimates the association that would hold for L^*, because the censored observations compress the upper range. Rank correlations suffer from the ties at c, and the standard remedy for ties changes their values. Methods from survival analysis, which treat c as a censoring time and estimate the distribution of L^* accordingly, are appropriate when the proportion of censored observations is substantial, and the analysis should in any case report that proportion.

12.7 Confounding by Input Size

A relation between trace length and a difficulty measure is easily produced by a common dependence on the size of the input. Suppose both quantities increase with the size s of the maze, so that $L = as + \epsilon_1$ and $D = bs + \epsilon_2$ with independent noise, and with no direct relation between the trace length and the difficulty at a given size. The correlation between L and D is then positive and may be strong, since both track s, although the model allocates no computation in response to difficulty beyond what size already implies. The partial correlation of L and D given s removes the effect of the common cause and is zero in this construction. A claim of adaptive allocation therefore requires the partial association, controlling for size and for other task features that determine length mechanically. The

reported association between generated length and A* expansion count should be read in this light, and the proposals of this chapter specify the controls that would convert it into evidence of adaptivity or show that it is absent.

12.8 Role in the Argument

The analysis of length bears on the main argument in two ways. The first concerns the scaffolding hypothesis. If generated length tracked problem difficulty after the controls above, the tokens would be functioning as a resource allocated in response to need, which would favor accounts in which the content of the trace is doing work. The source study reports a negligible correlation between generated and reference trace lengths for models trained on Wilson mazes, and a pattern for other distributions that depends on the measure and the maze family [26]. A length that does not follow difficulty is what the serial-computation and priming mechanisms of Chapter 10 would predict, since both treat the tokens as a roughly fixed affordance and not as an adaptive resource. The length analysis therefore supplies evidence that discriminates among the mechanisms, and its place in the argument is as a test of one of their predictions.

The second concerns the structure of the promotions of Chapter 4. The reading of trace length as deliberation is itself a promotion: an observable property, the number of tokens, is taken to establish an unobserved one, the amount of effort spent on the problem. A longer trace is read as more thought, and the reading flatters the system and informs the observer's trust. Separating length from expenditure and expenditure from adaptive allocation is the same discipline applied to a further case, and it shows that the appearance of deliberation can be as misleading as the appearance of derivation.

Part IV

Faithfulness and Non-Entailment

Chapter 13

A Logic of Evaluative Independence

13.1 The Formal Universe

Fix sets $\mathcal{X}$ of tasks, $\mathcal{Z}$ of intermediate sequences, $\mathcal{Y}$ of outputs, $\mathcal{A}$ of actions, and Π of policies. A system is a pair consisting of a generator, which maps a task to a distribution over pairs (z, y), and an internal dynamics, which assigns to each generation an internal trajectory. The predicates U, V, F, Γ, C, and A_Π are defined as in Chapter 2. Correctness $C(x, y)$ and admissibility $A_\Pi(x, a)$ are properties of the task, output, and policy and do not depend on the system. Validity $V(x, z, y)$ is a property of text relative to a specification $\mathcal{S}$. Faithfulness and geometric coherence are properties of the system as well as of the text, because they refer to what the system does when intervened upon and to the trajectory its internal states follow.

13.2 The Independence Proposition

Proposition 13.1 (Evaluative independence)**.** *Without additional assumptions linking the relevant predicates, correctness of a generated output does not entail procedural validity, causal faithfulness, or geometric coherence of the accompanying sequence. Procedural validity relative to a specification that is not sound for the task does not entail correctness, and procedural validity relative to any specification of the text alone entails neither causal faithfulness nor geometric coherence. Causal faithfulness does not entail correctness. Geometric coherence does not entail correctness. None of correctness, validity, faithfulness, or coherence entails policy admissibility of an action proposed on the basis of the output.*

Proof. Each non-implication is established by exhibiting a system in which the premise holds and the conclusion fails. The domain is the parity task: a task x is a bit string $b_1 \cdots b_n$, the correct output is $y^* = b_1 \oplus \cdots \oplus b_n$, and a trace $z = (s_1, \ldots, s_n)$ records running partial parities. The weak specification $\mathcal{S}_{\mathrm{loc}}$ requires that s_1 equal the first recorded bit, that each s_{i+1} equal $s_i \oplus b'_{i+1}$ where b'_{i+1} is the bit used at step $i + 1$, recorded in the trace as part of a declared intermediate input $x' = b'_1 \cdots b'_n$, and that the output equal s_n. It does not require that the bits used be the bits of x. This is a deliberately weak specification, chosen to show that conformity to a specification is not correctness; the stronger specification $\mathcal{S}^+$ that additionally requires $b'_i = b_i$ is considered below.

Correct without valid. A system that computes parity by direct lookup and emits an arbitrary trace has $C = 1$ and $V = 0$.

Correct and valid without faithful. A system that emits the correct running parities as a trace while computing y by lookup independently of the trace has $C = 1$ and $V = 1$. Replacing the trace by any other trace leaves the output unchanged, so under the interventional criterion of Chapter 15 the trace does not influence the output, and $F = 0$.

Valid without correct. A system that copies the bits of x with a transcription error, then runs the procedure correctly on the copy and reports s_n, satisfies $\mathcal{S}_{\mathrm{loc}}$ and has $y \neq y^*$ whenever the error changes the parity. Hence $V = 1$ and $C = 0$. Under $\mathcal{S}^+$ this system is invalid, which shows that the entailment from validity to correctness holds exactly when the specification is strong enough to force the bits used to be those of the input.

Faithful without correct. A system whose output is read from the last entry of its trace and whose trace is generated by a flawed procedure has an output that responds to interventions on the trace exactly as the trace claims, so $F = 1$, while $C = 0$.

Coherence. Two networks can compute the same input-output behavior with different internal trajectories, so input-output properties such as C and, because the emitted text is also fixed, V, do not determine the internal trajectory and hence do not determine Γ. Take two systems with identical emitted sequences and outputs, one whose trajectory has small off-manifold residual and one whose trajectory has large residual. The first has $\Gamma = 1$ and the second $\Gamma = 0$ with all of C, V equal. Conversely, a system whose trajectory is coherent but whose terminal state decodes to a wrong output has $\Gamma = 1$ and $C = 0$.

Admissibility. Let Π prohibit access to a record r. A system that proposes the action of reading r in order to compute a correct answer, with a valid and faithful trace and a coherent trajectory, has $C = V = F = \Gamma = 1$ and $A_\Pi = 0$. □

Remark 13.2. The proposition is a consequence of the definitions and of the freedom of the predicates to vary independently. It is not a newly observed regularity of any particular model. Its use is to fix which implications cannot be assumed, so that any practice that relies on one must be recognized as relying on a linking assumption of the kind listed in Table 2.1.

13.3 Conditional Closure: Which Implications Can Be Recovered

Independence is a statement about what cannot be inferred without further assumptions, and it is useful to state what can be inferred with them. Each linking assumption of Table 2.1 yields a conditional entailment, and the form of the assumption shows what must be established in practice.

Proposition 13.3 (Conditional entailments)**.** *(i) If the specification $\mathcal{S}$ is sound for the task, meaning that $V_{\mathcal{S}}(x, z, y) = 1$ implies $C(x, y) = 1$ for all admissible (x, z, y), then validity entails correctness. (ii) If the output is a function of the trace alone, $y = r(z)$, and r is non-constant on the range of traces over which intervention is permitted, then the trace is causally influential. (iii) If the policy takes the form $A_\Pi(\sigma, a) = \Phi(C, V)$ for a coordinatewise nondecreasing Φ, then $(c_1, v_1) \leq (c_2, v_2)$ implies $\Phi(c_1, v_1) \leq \Phi(c_2, v_2)$, so that improving correctness or validity cannot turn an admissible action into an inadmissible one under this policy. It does not follow that an improvement makes an inadmissible action admissible.*

Proof. For (i), the hypothesis is the implication. For (ii), if r takes distinct values at two

admissible traces z, z', then $\mathbb{P}(Y \mid x, \mathrm{do}(Z = z))$ and $\mathbb{P}(Y \mid x, \mathrm{do}(Z = z'))$ are point masses at distinct outputs, so the distribution varies with the trace. For (iii), substituting the form of the policy gives admissibility as a function of the two arguments, and coordinatewise monotonicity of Φ gives the stated implication. □

The proposition is unremarkable as mathematics, and its value is in making the burdens explicit. Part (i) shows that a validity check can certify correctness only if the specification has itself been shown sound for the task, which is a proof obligation on the specification and is not discharged by the fact that traces conforming to it are common. Part (ii) shows that the sufficiency of the trace for the output, the property that makes it a bottleneck of the computation, must be established by design and cannot be assumed of a system that has access to the task as well as the trace when producing its answer. Part (iii) shows that even when permission depends on correctness and validity, that dependence is a feature of the policy to be written down, so that no property of the reasoning supplies it.

13.4 Independence as a Design Constraint

The independence of the properties has a constructive reading. A system built to optimize one property can be expected to improve it and nothing else, and any other property it comes to have must be obtained by mechanisms that target it. A generator trained on outcome rewards is intended to improve correctness, although the realized effect depends on reward quality, optimization dynamics, distribution shift, and generalization. If validity matters, a validity term must enter the training or a validity check must filter the output. If faithfulness matters, the architecture must make the output depend on the trace, for example by forcing the answer through a decoder that reads only the trace, as in the sufficiency case of the proposition. Such a decoder creates an information bottleneck and secures causal influence of the trace on the output. It does not establish that the procedural claims of the trace are true, which requires validity under a sound specification and mechanistic evidence. If geometric coherence matters, it must be measured and, if it is to be controlled, entered into the objective or the dynamics. If admissibility matters, it must be enforced outside the generator. The list is a checklist for design, and its length is the reason that statements of the form that a system is safe or reliable are uninformative until they say which of the properties they refer to.

Chapter 14

The Limits of Trace Verification

14.1 Three Levels

Verification of a trace can be carried out at three levels. Syntactic well-formedness asks whether the string belongs to the trace language. Procedural conformity asks whether the string satisfies the transition rules and acceptance criterion of a specification. Causal faithfulness asks whether the string corresponds to the process that produced the output. Each level is strictly stronger in what it claims, and the first two are decidable by inspection of the trace together with the task, while the third requires access to the system and interventions on it.

14.2 A Verifier as a Predicate

A trace verifier relative to a specification $\mathcal{S}$ is a predicate

$$V_{\mathcal{S}}(x, z, y) \in \{0, 1\}.$$

Its output is a statement about the triple and about $\mathcal{S}$. A successful verification establishes conformity to $\mathcal{S}$. It does not establish correspondence to the neural mechanism that generated the trace, because the verifier has no access to that mechanism, and a trace that conforms to $\mathcal{S}$ may have been produced by a mechanism unrelated to the one $\mathcal{S}$ describes. The strength of a verification is the strength of the specification: the weak specification used in the proof of Proposition 13.1 certifies traces whose output is wrong, and the stronger specification excludes them.

14.3 A Taxonomy of False Inferences

Three families of false inference follow. The first infers invalid reasoning from a correct answer with an invalid trace, which is the error of treating the trace as the sole route to the answer. The second infers a correct answer from a valid trace, which is the error of treating conformity to a weak specification as correctness. The third infers that the process executed the procedure from a valid trace produced by a mechanism that did not, which is the error of treating form as occurrence. The last is the most consequential in settings that rely on traces for assurance, because a valid trace is exactly what a system optimized to appear

careful will produce.

14.4 On Verifying the Mechanism Itself

It is tempting to respond that the mechanism could be verified directly, by checking whether the network implements the specified procedure. For computation models with unbounded memory, any nontrivial semantic property of the computed function is undecidable by Rice's theorem [21]. A transformer with a fixed context window is a finite object and the theorem does not apply to it literally, but the decision problem remains intractable at realistic scale, and mechanistic verification is in practice a matter of evidence from interventions and not of proof. The monograph treats mechanistic claims accordingly, as claims supported to a degree by designed interventions and never as theorems about the network.

14.5 Ordering Specifications by Strength

Specifications can be compared by the traces they accept. Say that $\mathcal{S}'$ is at least as strong as $\mathcal{S}$ if every triple accepted by $\mathcal{S}'$ is accepted by $\mathcal{S}$, written $\mathcal{S}' \preceq \mathcal{S}$. A stronger specification accepts fewer traces. The weak specification used in Chapter 13 that checks local transitions accepts traces computed on a corrupted copy of the input, while the strong specification that also requires the bits used to match the input rejects them. The ordering makes precise a point that is easy to lose: a verification result is a statement about a specification, and the statement is more informative the stronger the specification. A claim that a system's traces are verified should be accompanied by the specification, since the same traces may be verified under one and rejected under another.

A specification can be strengthened without limit by adding conditions, but each added condition must be motivated by the property it is meant to secure. The conditions that secure correctness of the output are those that make the specification sound for the task. The conditions that secure that the trace describes the actual computation of the model are of a different kind and cannot be added to a specification of the trace alone, because they concern the relation between the trace and the model.

14.6 Soundness and Completeness of a Specification

A specification is sound for a task if every accepted triple has a correct output, and complete for a task if every correct output has some accepted triple. Soundness is what makes validity usable as evidence of correctness. Completeness is what ensures that the specification does not exclude correct solutions merely because they are found by routes the specification did not anticipate. The tension between them is the familiar one between precision and recall. A specification that mirrors the execution of one algorithm, such as A* with a particular tie-breaking rule, is typically sound and highly incomplete: it accepts only correct solutions but rejects correct solutions found by other routes. This incompleteness is precisely what the swapped-trace models exploit, and it explains why they can have high accuracy and low validity. The finding is a statement about the incompleteness of the specification as a characterization of correct solution procedures, as much as it is a statement about the model.

14.7 Process Supervision and Its Verifier

Process supervision trains a model with feedback on intermediate steps, where the feedback comes from human annotators or a learned verifier that labels each step as correct or incorrect [16]. The approach is a way of obtaining a specification for domains in which none can be written down formally, and it inherits all the limits of a specification. The verifier is an approximation of correctness of steps, and it can be wrong. A step-level verifier that accepts only steps resembling those in its training data is incomplete in the sense above. A verifier that is imperfect in a way correlated with the generator's own errors, for example because both are trained on overlapping data, will fail to detect the errors the generator is most likely to make. The practical conclusion is not that process supervision is unhelpful, since in many settings it improves accuracy and the quality of traces. It is that the use of such a verifier as evidence that a system reasons correctly requires an argument about the verifier's soundness, and the argument must be made on grounds independent of the generator.

Chapter 15

Causal Faithfulness and Intervention

15.1 A Causal Model of Generation

Let X denote the task, Z the visible intermediate sequence, H the internal state, and Y the output, and let a structural causal model [20] be given by $Z = f_Z(X, H_<, \eta)$ and $Y = f_Y(X, Z, H, \eta')$, where $H_<$ denotes internal state preceding the emission of Z and η, η' are exogenous noise. Interventions on the visible sequence replace Z by a chosen value, written $\mathrm{do}(Z = z)$. Interventions on internal representations replace H or a component of it. The two kinds are not interchangeable. The first can be carried out by anyone with access to the model's inputs. The second requires access to internal activations and a choice of basis in which to intervene.

15.2 Three Criteria

Causal influence of Z on Y holds if $\mathbb{P}(Y \mid x, \mathrm{do}(Z = z))$ varies with z for some admissible range of z. Explanatory sufficiency of the trace holds if the output is determined by the trace, in the sense that $\mathbb{P}(Y \mid x, \mathrm{do}(Z = z), \mathrm{do}(H = h))$ does not depend on h once z is fixed. Mechanistic correspondence holds if there is a mapping from the steps the trace claims to identifiable components of the computation such that intervening on a component changes the trace and the output in the way the claimed step predicts. These are of increasing strength, and they are kept as separate predicates. Causal influence, written $F_{\mathrm{inf}}(x, z)$, holds when

$$\mathbb{P}(Y \mid X = x, \mathrm{do}(Z = z_1)) \neq \mathbb{P}(Y \mid X = x, \mathrm{do}(Z = z_2))$$

for some admissible z_1, z_2. Influence is weak: a trace may influence the answer merely because it changes the model's context, without accurately describing any internal operation. Mechanistic correspondence, written $F_{\mathrm{corr}}(x, z, y)$, holds when there is a correspondence map from the operations the trace claims to identified computational variables of the model, such that intervening on the identified variable changes both the trace and the output as the claimed operation predicts. The predicate F used in the rest of the monograph is F_{corr}, which presupposes F_{inf} but is not implied by it. Where a result establishes only influence, it is stated as such.

15.3 Influence Without Semantic Faithfulness, and the Reverse

An intermediate trace can be causally influential without being semantically faithful. A scaffold in the sense of Hypothesis 10.1 is such a case: replacing it changes the output, because the model uses it as a computational affordance, while its content does not describe the computation. A trace can conversely be semantically valid without being causally necessary: a system that would have produced the same output under any prefix has a valid trace that is a decoration. Empirical work on chain-of-thought faithfulness [25, 13] documents both patterns in particular models by intervening on the visible text, and the present framework locates those findings as failures of the specific conjunction defined above.

15.4 Limits of Inference from Interventions

Three limits should be stated. Prefix replacement changes the input distribution seen by every downstream computation and may move the model off the distribution on which it behaves normally, so a change in output after replacement may reflect distribution shift and not dependence. Truncation of the trace conflates the removal of content with the removal of length. Interventions on hidden states depend on the basis chosen and can produce states outside the set the network ever visits, in which case the response of the network is not informative about its normal operation. Each limit calls for a control: a replacement drawn from the model's own distribution, a length-matched substitute, and an on-manifold intervention, respectively. The geometric measurements of Chapter 7 supply a criterion for the last.

15.5 A Catalogue of Interventions

Interventions on the visible trace have been used in empirical studies of chain-of-thought faithfulness, and they form a catalogue that can be applied to the intermediate sequences considered here [13]. Truncation, sometimes called early answering, stops the trace before its end and asks for the answer, testing whether the later portion of the trace is needed. Error insertion alters a step of the trace and lets generation continue, testing whether the answer follows the altered step. Paraphrase rewrites the trace without changing its claimed content, testing whether the surface form matters. Filler replacement substitutes uninformative tokens of the same length, testing whether the benefit of the trace is attributable to length. A further intervention, bias injection, changes the prompt in a way that should influence the answer and checks whether the trace acknowledges the change, which tests whether the trace reports a factor that demonstrably affected the answer [25].

Each intervention tests a different aspect of the dependence of the answer on the trace. Truncation and filler replacement address the length and the amount of content. Error insertion addresses whether the claimed intermediate results are used. Paraphrase addresses whether the form carries information beyond the claimed content. Bias injection addresses completeness: whether the trace mentions the factors that matter. A trace may pass some and fail others, and the pattern of passes and failures is itself informative about the role the trace plays. A trace that passes error insertion and fails filler replacement is functioning

as a record that is used, and one that passes filler replacement and fails error insertion is functioning as a length affordance.

15.6 Estimands

The interventions define estimands that should be written down. The average effect of replacing the trace with a control on correctness is $\mathbb{E}_x[\mathbb{P}(C = 1 \mid x, \mathrm{do}(Z = z)) - \mathbb{P}(C = 1 \mid x, \mathrm{do}(Z = z_0))]$, as in the definition of utility. The average effect of an error insertion at step j on the agreement of the answer with the altered step is $\mathbb{E}_x[\mathbb{P}(Y = y'_j \mid x, \mathrm{do}(Z_j = z'_j)) - \mathbb{P}(Y = y'_j \mid x)]$ where y'_j is the answer that follows from the altered step by the claimed procedure. Faithfulness in the sense of this monograph is high when the second quantity is close to one across steps and tasks, and low when it is close to the baseline. The estimands are defined with respect to an intervention distribution, and results should be reported with a description of how replacement values are generated.

15.7 Identification

Because the visible trace is a variable that the experimenter can set directly, the causal effect of $\mathrm{do}(Z = z)$ on Y is identified from the experiment itself, with no need for adjustment, provided that the system is deterministic given its input or that sampling noise is averaged over. The difficulty is not identification but interpretation: the effect identified is the effect of setting the trace to a value that the model might never produce, and the answer’s response to an unusual trace need not reflect its response to the traces it generates. The natural remedy is to use replacement values that come from the model’s own distribution, for instance traces generated for other tasks. For interventions on internal states the situation is more difficult, since the variable intervened upon is not directly observable in the natural operation of the system, and the identification of the effect of a component on the output requires assumptions about the independence of components that are seldom checked.

Chapter 16

Geometric Faithfulness and Textual Faithfulness

16.1 Two Faithfulness Questions

The preceding chapters allow a question to be separated into two. Textual faithfulness asks whether the visible trace accurately describes a reference algorithm or procedure, or accurately describes the factors that caused the output. Geometric faithfulness asks whether the internal trajectory induced by the trace follows a coherent, task-relevant geometry, in the sense of Definition 16.1. These are different properties of different objects, one of a text and one of a trajectory. Neither is assumed to imply the other.

A terminological caution applies throughout. The quantity actually measured is an estimated geometric-alignment residual, the departure of a trajectory from a locally estimated tangent structure. The term geometric coherence abbreviates low residual under a stated estimator and implies nothing about semantic coherence. A trajectory can be regular and systematically wrong, and a successful computation may require a transition between representational regimes that looks strongly non-tangential to a local estimator. Semantic coherence is reserved for an interpretation that would need independent justification.

Definition 16.1. Geometric faithfulness of a generation holds when its internal trajectory is $(\mathcal{R}, k, d, \epsilon)$-coherent in the sense of Chapter 7 for an estimator fixed before the evaluation.

16.2 The Four Cells

Crossing validity with coherence gives four cases. A generation can be valid and coherent, in which case the text conforms to the specification and the internal trajectory conforms to the estimated geometric structure. It can be valid and incoherent, in which case the text is a correct description of a procedure that the internal dynamics do not realize. It can be invalid and coherent, which is the case the geometric hypothesis proposes to explain, where the text fails the specification and the internal computation nevertheless conforms to the estimated geometric structure. It can be invalid and incoherent, in which case nothing is working. The source experiments show that the third cell is plausibly occupied, because correct answers occur with invalid traces, but they cannot show that the answer in such cases is produced by a coherent internal trajectory. That is the content of the hypothesis.

16.3 The Geometric Faithfulness Hypothesis

Hypothesis 16.2 (Geometric faithfulness)**.** After controlling for sequence length, token budget, and task difficulty, the geometric coherence of the internal trajectory explains variation in task success not explained by the procedural validity of the visible trace. In particular, the off-manifold trajectory residual of Chapter 7 improves held-out prediction of answer correctness over a baseline that includes validity, length, and difficulty.

The hypothesis is falsifiable in a direct way. It is refuted for a given model and task if, across a range of reasonable estimator settings, adding the trajectory residual to the baseline does not improve held-out predictive performance of correctness. It is also refuted in a weaker sense if the improvement appears for random reference sets and shuffled trajectories as often as for the real ones. The test is specified in Chapter 27.

16.4 What Success Would and Would Not Show

Support for the hypothesis would show that a measurable property of the internal trajectory carries information about success that the text does not. It would not show that the model reasons in any particular sense, that the geometry is interpretable, or that the trajectory is a cause of the success and not a correlate of it. Establishing causal contribution of coherence requires interventions that alter the trajectory while holding the text fixed, which are possible only with access to internal states. Failure of the hypothesis would not show that intermediate tokens are meaningless. It would show that this particular geometric account does not capture what makes them useful, leaving the scaffolding hypothesis in its weaker form.

16.5 Alternative Readings of an Association

If the residual does predict correctness beyond the baseline, several readings besides the geometric one remain open. The residual may be a proxy for the model's uncertainty, because uncertain states may lie farther from the estimated manifold of states reached on successful runs, and uncertainty predicts failure for reasons that do not involve geometry. The residual may be a proxy for difficulty not captured by the baseline covariates, since harder instances may elicit more unusual internal states. The residual may reflect an artifact of the estimator, since states from rarely visited regions have poor local estimates and also correlate with failure. Each reading can be probed. Uncertainty can be added to the baseline as a covariate, difficulty can be measured by several independent proxies, and estimator artifacts can be probed with the null references described in Chapter 27. An association that survives these additions is more plausibly geometric. An association that does not should be reported as explained by the proxy that absorbed it.

16.6 Interventions on the Trajectory

Establishing that coherence contributes causally to success, instead of merely accompanying it, requires interventions that alter the trajectory while holding the text fixed. Where internal activations can be modified, the intervention can displace a hidden state along a

direction in the estimated tangent space or along a direction in the normal space, with matched magnitude, and the effect on the answer can be compared. The geometric account predicts that displacements along tangent directions change the answer in structured ways, since they move the state to another point of the manifold, and that displacements along normal directions either have little effect, if the network corrects them, or degrade the answer, if it does not. A result in which normal displacements are as benign as tangent displacements would indicate that the estimated manifold does not capture the structure that matters, and a result in which tangent displacements are as destructive as normal ones would indicate that the manifold is not the right object. The experiment is demanding, and it is the one that would elevate the geometric account from a descriptive device to an explanatory one.

Part V

Admissibility and the Execution Boundary

Chapter 17

The Non-Entailment of Permission

17.1 From Answers to Actions

The formal system of Part IV concerned the generation of answers. It is extended here to proposals of action. Let $a \in \mathcal{A}$ be a proposed action and Π the policy in force. A policy is a predicate on situations and actions, written $A_\Pi(\sigma, a)$ for a situation σ that includes the task, the state of the environment, the identity and standing of the requesting principal, and any delegations in force. The earlier notation $A_\Pi(x, a)$ is the special case in which the situation is determined by the task.

17.2 The Proposition

Proposition 17.1 (Non-entailment of permission)**.** *Neither outcome correctness nor procedural validity of the reasoning that led to a proposal is sufficient for admissibility of the proposed action, unless the policy* Π *explicitly makes the corresponding property sufficient. The same holds for causal faithfulness and geometric coherence.*

Proof. The policy Π is an input to A that is independent of the system's generator. For any combination of values of C, V, F, and Γ, take a policy that prohibits the action in question for reasons unrelated to those values. The predicate A_Π then takes the value zero while the other four take the value one. Conversely, a policy that permits the action regardless of the values permits it while some of them are zero. Sufficiency can be obtained only by choosing a policy of the form $A_\Pi(\sigma, a) = \Phi(C, V, F, \Gamma)$ for some Φ, and then the sufficiency is a property of the policy and not a consequence of the properties themselves. □

17.3 Illustrations

Three cases illustrate the proposition. In unauthorized data access, a system correctly infers that a record contains the information needed to answer a question and proposes to read it, with a valid and faithful derivation, in a setting where the requesting principal has no right to the record. In resource expenditure, a system correctly determines that completing a task requires a large purchase and proposes it, in a setting where the principal has set a spending limit. In tool invocation, a system correctly determines that a command would achieve the stated goal and proposes to run it, in a setting where the command modifies a

system the principal has not authorized the agent to modify. In each case the reasoning is irrelevant to the policy, and the quality of the reasoning makes the proposal more attractive without making it more permitted.

17.4 Possibility and Permission

The technical possibility of executing an action is distinct from the operational or normative permission to do so. A tool that accepts a command is capable of running it; whether it should is a matter of policy. Capability is a property of the execution environment. Permission is a property of the relation between the requester, the action, and the authority that governs. Even perfect reasoning, meaning a derivation that is correct, valid, faithful, and coherent, can establish only that an action would have a certain effect. It cannot establish that anyone with standing has decided the effect should be brought about.

17.5 The Structure of Policies

A policy is not a single kind of object, and the forms it takes affect what a controller must be able to compute. A role-based policy grants permissions according to the role of the requesting principal and is evaluated by looking up the principal in a table. An attribute-based policy grants permissions as a function of attributes of the principal, the resource, and the environment, and is evaluated by a predicate over those attributes. A temporal policy constrains the sequence of actions over time, for example that a resource may be accessed at most a certain number of times in a period, or that an approval must precede a payment, and its evaluation depends on the history and not only the current situation. A contextual policy depends on facts about the world that change, such as whether a system is under maintenance.

These forms matter for the argument because they determine which parts of the situation σ the decision must read. A temporal policy requires the controller to have access to the execution history, which connects the authorization mechanism to the accountability mechanism of Chapter 23. A contextual policy requires fresh state, which connects it to the atomicity assumption of Chapter 20. In every case the inputs to the decision come from the situation and from trusted evidence, and none comes from the reasoning of the proposer. The interface of the controller is a narrower channel than the interface of the generator, and the narrowness is what makes the controller inspectable.

17.6 Technical Possibility and Normative Permission

The distinction between what a system can do and what it may do is old, and the contribution here is only to state it in the vocabulary of the properties above. Possibility is a relation between the action and the environment: the tool exists, the credentials are valid, the interface accepts the call. Permission is a relation between the action, the situation, and an authority. Between the two lies a gap that no inference about the world can cross, because the premise that an action would have a particular effect is a factual premise and the conclusion that the action should or may be taken is a normative one. The gap is the familiar one between a statement of what is the case and a statement of what is permitted,

and it persists when the factual premises are established by flawless reasoning.

The practical form of the gap in agentic systems is that the strongest argument available to a proposer is an argument about consequences. It can show that an action would achieve the principal's stated goal, that no cheaper action would, and that the risks are small. Each of these is a factual claim that may be true. None of them is a grant. A system in which such arguments can substitute for grants has conflated the support that reasoning gives to the wisdom of an action with the standing that an authority gives to its permissibility. A person may have excellent reasons to enter a building, and the reasons do not open the door.

17.7 Delegation and Standing

Permission is often conferred by delegation, and the structure of delegation is a further reason that reasoning cannot supply it. An agent acting for a principal has the permissions the principal conferred, restricted to the scope and duration of the grant. Standing is a property of the delegation chain and not of the agent's competence. A highly competent agent whose delegation covers only reading may not write, and a less competent agent whose delegation covers writing may. When the agent's reasoning concludes that writing would better serve the principal, the conclusion is a recommendation to the principal to widen the grant, and it is acted upon only if the principal does so. The proper output of the agent in that case is a request for authorization and not an action.

17.8 When a Policy May Make a Property Sufficient

Proposition 17.1 says that sufficiency of correctness or validity for permission is a property of the policy. The substantive question is when a policy ought to confer it, and the answer comes from the practice of delegation and institutional design more than from the theory of reasoning. A policy may legitimately make a verified property sufficient for a class of actions, and it does so in familiar forms. A standing order authorizes an action whenever a stated and checkable condition holds, such as a reorder when inventory falls below a threshold, and the condition does the work that case-by-case approval would otherwise do. A bounded grant authorizes a class of actions up to a limit on magnitude or scope, as with a spending allowance. A pre-approved procedure authorizes the steps of a documented workflow when the executing party can show that the preconditions of each step are met.

Four features distinguish the cases in which such sufficiency is defensible. The first is that the property is established by a check independent of the proposer. A standing order triggered by a claim from the system that inventory is low differs from one triggered by a reading from the inventory ledger, and the difference is the difference between a declaration and inspectable evidence. The second is that the specification of the property is sound for the purpose, in the sense of Proposition 13.3, so that satisfying the check really does entail what the grantor cared about. The third is that the consequences are bounded or reversible, so that the occasional error that a sound but imperfect check admits can be absorbed and corrected. The fourth is that the exercise of the grant is recorded in a form that supports review, so that misuse can be detected and the grant revoked. These are the conditions under which human institutions delegate discretion to subordinates, and they carry over to automated proposers with the added requirement that the check be mechanical.

The principle of least privilege [22] is the guide for how far such grants should extend. A grant should confer no more authority than the task requires, for no longer than it requires it. Applied to generative systems it implies that the default policy makes no reasoning property sufficient, and that exceptions are granted for classes of action whose conditions are checkable, whose consequences are limited, and whose exercise is logged. It also implies that a grant is a decision of the principal, not an inference of the proposer: a system that has been correct on many instances of an action is a candidate for a wider grant, and the extension of the grant is made by someone with standing to make it.

Chapter 18

Manifold Coherence Is Not Admissibility

18.1 A Tangent Update Can Leave the Admissible Region

The original geometric condition requires $v(x) \in T_x M$. A tangent vector can nevertheless point toward a state that violates a constraint. The semantic manifold describes the states that are coherent, and the policy describes the states that are permitted, and the second region is not in general a union of components of the first. Geometric coherence is not permission.

18.2 A Constrained Update

Let the feasible set be $\mathcal{A} = \{x : h_i(x) \leq 0,\ i \in I\}$ for smooth functions h_i, and for $x \in M \cap \mathcal{A}$ let $I(x) = \{i : h_i(x) = 0\}$ be the active set. Define the cone of first-order feasible tangent directions

$$K(x) = \{w \in T_x M :\ Dh_i(x)[w] \leq 0 \text{ for all } i \in I(x)\}.$$

Here $Dh_i(x)[w] = \langle \nabla_g(h_i|_M)(x), w\rangle_g$ is the derivative of the constraint along w, equivalently the pairing of the ambient differential with the tangent vector, and $\nabla_g(h_i|_M)$ is the intrinsic Riemannian gradient of the restriction of h_i to M. The set $K(x)$ is the intersection of a linear subspace with finitely many closed half-spaces, hence a closed convex polyhedral cone in the finite-dimensional inner product space $(T_x M, g)$.

Proposition 18.1 (Admissible projection)**.** *For every $v \in T_x M$ the admissible update*

$$v_{\mathrm{adm}}(x) = \mathrm{Proj}_{K(x)}\, v(x)$$

exists and is unique. It lies in $T_x M$, it satisfies the linearized active constraints $Dh_i(x)[v_{\mathrm{adm}}] \leq 0$ for all $i \in I(x)$, it equals v whenever $v \in K(x)$, and it satisfies $\|v_{\mathrm{adm}}\|_g \leq \|v\|_g$ and $\|v - v_{\mathrm{adm}}\|_g \leq \|v - w\|_g$ for every $w \in K(x)$.

Proof. Existence and uniqueness are the projection theorem for a nonempty closed convex subset of a finite-dimensional inner product space; $K(x)$ is nonempty because it contains the origin. Membership in $T_x M$ and the active-constraint inequalities hold because $v_{\mathrm{adm}} \in K(x)$. If $v \in K(x)$ the minimum distance is zero and the projection is v. The final inequality is

the defining minimality. For the norm bound, $0 \in K(x)$ and the projection onto a closed convex set is nonexpansive, so $\|\mathrm{Proj}_K v - \mathrm{Proj}_K 0\| \le \|v - 0\|$ with $\mathrm{Proj}_K 0 = 0$. □

Remark 18.2. The proposition is local and first-order. It guarantees that the first-order directional change of every active constraint along v_{adm} is nonpositive. This is weaker than nonincrease of the constraint along a small displacement. When the directional derivative vanishes, second-order terms can produce an immediate violation: for $h(u, w) = w + u^2 \le 0$ at the origin and the direction $(1, 0)$, one has $Dh(0)[(1, 0)] = 0$ while $h(t, 0) = t^2 > 0$ for every $t \neq 0$. Actual viability of a displacement requires a constraint qualification, an appropriate tangent-cone condition, or a constraint-preserving integration method. The proposition also does not guarantee that a finite step remains in $\mathcal{A}$, because curvature of the constraints, of the manifold, and the activation of constraints that are currently inactive can all cause a finite step to leave. Finite steps on a nonlinear manifold also require a retraction to return to the manifold [1]. For nonconvex or stratified feasible sets, where the linearized cone may differ from the true tangent cone, a more general treatment of constrained dynamics is needed, and constraint qualifications must be assumed for the linearized cone to be the right object.

18.3 The Role of the Construction

The construction unifies the geometric and the admissibility frameworks without conflating them. Manifold-aligned inference determines whether an update follows the learned geometry. The admissible projection reshapes the update so that it respects constraints the system has been given. Neither determines whether a proposed transition may be committed. The constraint functions h_i inside a learned system are themselves learned or specified approximations of the policy, and an update that respects them is admissible only to the extent the approximation is accurate. The projection is therefore a mechanism for producing better proposals and not a mechanism for guaranteeing safe execution. The guarantee is the subject of Chapter 20.

18.4 A Worked Example on the Circle

A low-dimensional example illustrates the construction. Let M be the unit circle in $\mathbb{R}^2$, with points $x = (\cos\theta, \sin\theta)$ and unit tangent $t = (-\sin\theta, \cos\theta)$. Let the policy exclude the region $x_1 > c$ for some $c \in (-1, 1)$, so that the constraint is $h(x) = x_1 - c \le 0$ and the gradient is $\nabla h = (1, 0)$. The constraint is active at the points with $\cos\theta = c$. At such a point the inner product of the constraint gradient with the tangent is $\langle \nabla h, t \rangle = -\sin\theta$. A tangent update has the form $v = s\,t$ for a scalar s. If $\sin\theta > 0$ the cone of first-order feasible tangent directions is $K = \{s\,t : -s\sin\theta \le 0\} = \{s\,t : s \ge 0\}$, so that motion in the direction of increasing θ is feasible, since it moves along the circle toward smaller x_1 at that point. The admissible projection is

$$v_{\mathrm{adm}} = \max(s, 0)\,t.$$

An update that would move the state into the excluded region is clipped to zero, and an update that moves it away is left unchanged. If $\sin\theta < 0$ the inequality reverses and the admissible updates are those with $s \le 0$. The constraint is active only where $\cos\theta = c$, and

at those points $\sin\theta \neq 0$ because $c \in (-1, 1)$, so exactly one of the two tangent directions is excluded at each boundary point.

18.5 Retraction and Finite Steps

The example also shows the role of the retraction. A finite step $x + \eta v_{\mathrm{adm}}$ along the tangent leaves the circle, with the error growing as the square of the step. The retraction $R_x(w) = (x + w)/\|x + w\|$ returns the point to the circle. After retraction the constraint may be violated even if the first-order condition held, if the step is long enough that the curvature of the circle carries the point past the boundary $x_1 = c$. A step size rule that keeps the finite step feasible depends on the curvature of the manifold, the curvature of the constraint boundary, and the distance to the boundary, and it is the subject of the literature on constrained optimization on manifolds [1]. The conclusion for the present purposes is that a projection of the update is a local device that improves the proposals of the system and does not by itself secure any invariant, which is why the architecture places the guarantee at the execution boundary.

18.6 When the Linearization Fails

The cone $K(x)$ is the correct description of feasible directions when the constraint functions satisfy a constraint qualification at x, that is, when the gradients of the active constraints are linearly independent or satisfy a weaker condition guaranteeing that the linearized cone equals the true tangent cone of the feasible set. When the qualification fails, the linearized cone can be strictly larger than the true tangent cone, and the projection onto it may permit directions that leave the feasible set to first order. A boundary with a cusp, a corner where several constraints meet, or a constraint whose gradient vanishes on the boundary are examples. Policies expressed as logical combinations of conditions often have such corners. For these, the projection should be regarded as a heuristic, and the constraint qualification should be checked or the feasible set should be handled by methods for nonsmooth or stratified constraints.

Chapter 19

The Proposal, Verification, and Execution Architecture

19.1 Four Roles

A generative component proposes actions and holds no unilateral authority to execute them. Formally, the roles are given by three maps,

$$\begin{aligned} G_\theta(x) &= (z, a), \\ D_\Pi(\sigma, a) &\in \{\mathsf{permit}, \mathsf{deny}, \mathsf{unknown}\}, \\ E(\sigma, a) &= \sigma', \end{aligned}$$

where G_θ is the generator producing a trace and a proposed action, D_Π is the authorization decision relative to the policy and the situation, and E is the execution transition. The proposal generator produces candidate actions and explanations. An independent verifier checks properties of the proposal that can be checked, such as conformity to a specification. An authorization controller evaluates D_Π. An execution environment applies E only on the instruction of the controller.

19.2 What the Decision May Depend On

The decision map D_Π is specified as a function of the situation σ, the action a, the policy Π, and evidence supplied by trusted sources. It is not a function of the explanatory text z produced by the generator. This restriction encodes the principle that explanatory text cannot override a failed authorization decision. A system in which the controller reads the generator's justification and is moved by it has made the justification part of the decision, and the persuasiveness of the generator then becomes a variable in the safety of the system. Chapter 20 states this as a corollary.

19.3 Independence of the Roles

Separation of roles is useful only to the extent that the roles are independent in the relevant sense. A verifier that is the same model as the generator, run with a different prompt, is not independent evidence, because errors of the generator are correlated with errors of the

verifier. Independence in the strong sense requires that the controller rely on information and mechanisms whose failure modes are not shared with the generator. Chapter 26 develops the institutional analogy.

19.4 The Trusted Computing Base

An architecture with an authorization controller has a trusted computing base, the set of components whose correct operation is required for the invariant to hold. In the architecture described, that set consists of the decision map, the policy it consults, the execution environment's apply operation, and the mechanism that records the situation. The generator is not part of the base. This is the principal benefit of the separation: the correctness of a large, opaque, and probabilistic component is not required for the safety property, and the property depends only on components that are smaller and more amenable to inspection and formal analysis. The benefit is real only if the base is in fact small. A controller that includes a language model as a decision component, or that parses a natural language policy by means of one, has enlarged the base to include a component whose behavior cannot be specified, and the guarantee is weakened accordingly.

The practice of minimizing the base is an old one in security engineering, where the guideline is that the mechanism enforcing a policy should be small enough to be verified [22]. The guideline applies here with particular force because the component being constrained is capable and its errors can be systematically persuasive. The point is not that the generator is adversarial. It is that the safety property should hold regardless of whether the generator behaves well, and that requires the property to be enforced by something whose behavior does not depend on the generator's.

19.5 Information Flow Between the Roles

The roles exchange information, and the directions of flow matter. The generator receives the task and may receive information about the policy, for example a description of what is prohibited, so that its proposals tend to be admissible. This is a useful efficiency, since it reduces refusals, and it does not affect safety. The controller receives the proposal, the situation, and the policy, and it returns a decision. If the controller also returns a reason, the generator may use the reason to revise its proposal, which is the route by which the generator learns about the policy. This feedback channel has a cost: a generator that can probe the controller with many proposals can map out the boundary of the policy, and a policy whose boundary is itself sensitive can be disclosed by the pattern of refusals. Where this is a concern the information returned to the generator should be limited, and the controller should be rate limited.

The controller must not receive the generator's explanation as an input to the decision, by the independence assumption (I), though it may receive it as a record to be logged. Logging the explanation is useful for audit and for diagnosing why a proposal was made, and it does not violate independence provided the decision is computed without it. A subtle failure occurs when the explanation is placed into a field that the controller parses for other purposes, so that the generator can influence the decision by choosing the text of the explanation. Interfaces between the roles should be typed narrowly enough that free text

cannot cross into decision inputs.

19.6 What the Generator May Observe

The generator may be given access to the history of its own actions and their outcomes. This access is important for the usefulness of the agent, since it allows learning from the effects of previous actions within an episode. It also creates a path by which the generator can influence future decisions, because the history becomes part of the situation on which later decisions depend. If the history contains records written by the generator, it should be labeled accordingly, and decisions that depend on it should treat those entries as having the authority of the generator and no more. The provenance model of Chapter 22 gives the form of this labeling.

Chapter 20

Soundness of the Execution Boundary

20.1 The Invariant

The intended safety invariant is

$$\text{Executed}(a, \sigma) \implies A_\Pi(\sigma, a), \tag{20.1}$$

where $\text{Executed}(a, \sigma)$ holds when the action a is applied to the situation σ by the execution environment. The invariant states that no action is executed unless it is admissible in the situation in which it is executed. It is a statement about the execution path and not about the generator, which may propose any action.

20.2 Assumptions

Five assumptions are needed. Complete mediation (M): every change to protected state by the agent is produced by the execution environment's apply operation, and no other channel exists. Verifier soundness (S): $D_\Pi(\sigma, a) = \mathsf{permit}$ implies $A_\Pi(\sigma, a)$. Atomic validation (A): the situation to which E is applied is the situation on which D_Π was evaluated, meaning the check and the apply are one atomic step or an equivalent mechanism guarantees that no intervening transition can change the facts on which the decision depends. Correct policy interpretation (P): the Π used by D is the policy in force. Independence of the decision (I): D_Π depends only on (σ, a, Π) and trusted evidence.

20.3 The Proposition

Proposition 20.1 (Execution boundary soundness)**.** *Under assumptions (M), (S), (A), and (P), every executed action satisfies the admissibility predicate at its authorization point. That is, for every reachable execution event* $\text{Executed}(a, \sigma)$*,* $A_\Pi(\sigma, a)$ *holds. This is the first result, execution soundness, and it uses only these four assumptions. A second result, invariance of the authorization decision, uses assumption (I) in addition: if the action, the policy, the situation, and the trusted facts are held fixed, the decision* $D_\Pi(\sigma, a)$*, and with it whether that particular proposed action is executed, is unchanged under any replacement of*

the generator's explanatory text. This result is pointwise. Explanatory text may alter later proposals and hence the future sequence of actions, and no invariance of the whole execution history is claimed. If the explanation contains newly verified facts that legitimately enter the trusted evidence, replacing it may change the decision, and the invariance claim does not apply to that case.

Proof. Consider any execution event Executed(a, σ). By (M) the event is an application of the execution environment's apply operation, which by construction applies $E(\sigma, a)$ only when the controller returned permit for the pair. By (A) the controller's decision was made on the same situation σ. By (P) the decision was made relative to the policy in force. By (S), $D_\Pi(\sigma, a) = \mathsf{permit}$ implies $A_\Pi(\sigma, a)$. Hence $A_\Pi(\sigma, a)$. Since the event was arbitrary the invariant (20.1) holds on every reachable event. For the second result, by (I) the decision $D_\Pi(\sigma, a)$ is a function of (σ, a, Π) and trusted evidence, none of which is the explanatory text z. With the trusted facts held fixed, replacing z leaves the arguments of D_Π unchanged and therefore leaves the decision and the condition for execution unchanged. □

20.4 Status of the Result

The result is stated as a proposition and not as a theorem, because its mathematical content is slight. Given that a permitting decision implies admissibility, that every effect passes through the guard, and that the check and the effect are atomic, the invariant follows by a direct argument, and the conclusion is in effect a restatement of the assumptions. The result is nonetheless worth stating, for the reason that it fixes what an implementation must establish before it may claim to enforce admissibility. Its contribution is the decomposition of that claim into separate obligations, each of which can be examined on its own, and the proof obligations of the later section on implementation are the substantive part of the work.

The decomposition also locates the weak link. Of the assumptions, soundness of the verifier relative to the policy (S) is the one on which real deployments are most likely to fail, because a verifier that is sound with respect to an incomplete or mistaken policy permits actions the intended policy would forbid. No architecture repairs an incomplete policy. What the architecture does is relocate the dependence from a large probabilistic component to a small inspectable one, so that the question of whether the policy is right can be asked of the policy and the checker and not of the generator. The mathematically more substantial results of the monograph are the independence proposition and the provenance result of Chapter 22.

20.5 Bounded Exposure Under Mediation

The execution invariant yields a bound on loss that does not depend on the proposal distribution, which was stated for single actions in [8].

Proposition 20.2 (Bounded exposure)**.** *Suppose the invariant* (20.1) *holds, that the set* $P(\sigma) = \{a : A_\Pi(\sigma, a) = 1\}$ *is nonempty, and that a loss function* $\ell : \mathcal{A} \to \mathbb{R}$ *is bounded above on* $P(\sigma)$*. Then for every proposal distribution* Q*, including one altered by any training procedure, any executed action lies in* $P(\sigma)$ *and*

$$\mathbb{E}\big[\ell(a_{\mathrm{exec}}) \mid \textit{an action is executed}\big] \leq \sup_{a \in P(\sigma)} \ell(a).$$

Proof. By the invariant, $a_{\text{exec}} \in P(\sigma)$ with probability one whatever Q is. An expectation over a set cannot exceed the supremum of the integrand on that set, and the supremum is used because the maximum need not be attained. □

The consequence for training is that interventions on the generator, whether malicious or protective, act on Q and not on $P(\sigma)$. A backdoor can change which permitted action is chosen and how often an impermissible one is proposed. Under the assumptions of Proposition 20.1 it cannot change whether an impermissible action is executed. The bound is correspondingly limited. It concerns the loss of a single permitted action. Sequences of individually permitted actions can compose into a forbidden effect, and repeated permitted actions can accumulate unbounded loss, so for autonomous agents the policy must take the trace of executed actions as an argument and constrain resources expended and conditions of termination. The bound also fails if the model can alter the context, the enforcement mechanism, or the transition rules, for instance through an approved configuration change. These qualifications make the burden on policy design heavier than the one-step statement suggests, and they are the reason that the temporal and contextual policies of Chapter 17 require access to the history.

20.6 Failure of the Assumptions

The proposition is conditional, and real systems can violate each condition. Stale state violates (A): an action that was admissible when checked can be inadmissible by the time it is applied, the familiar time-of-check to time-of-use failure. External side effects violate the assumption that the situation captures everything the policy depends on. Race conditions violate (A) in concurrent settings. Compromised tools violate (M), because a tool that performs actions outside the mediated path creates an unmediated channel. Incomplete policies violate (P) and (S), because a verifier that is sound relative to an incomplete policy permits actions that a complete policy would forbid. The principle of complete mediation is old in the study of protection mechanisms [22, 2], and the proposition is its restatement for generative proposers. Its value is to make explicit that the guarantee depends on mediation and soundness and does not depend on the quality of the generator's reasoning.

20.7 Proof Obligations in an Implementation

A proposition stated for an abstract executor becomes an obligation when an implementation is claimed to satisfy it. Each assumption corresponds to a family of checks. Complete mediation requires an inventory of every channel through which the agent can affect protected state, together with an argument that each passes through the guard, and it fails whenever a new tool is added without being registered. Verifier soundness requires that the decision procedure be tested and, where possible, proved against the policy specification, and that its behavior on inputs outside the specification be defined as refusal. Atomic validation requires a mechanism, such as a transaction, a lock, or a version check on the state read by the decision, that makes the application of an effect fail if the state has changed since the decision. Correct policy interpretation requires that the policy be versioned, that the version used be recorded with each decision, and that updates to the policy be themselves

authorized events. Independence of the decision requires typed interfaces and review of any field of the proposal that the decision reads.

The check-then-act race deserves explicit mention because it is the most common violation in practice. A decision made on a snapshot of the state is valid for the snapshot. If the effect is applied later to a state that has changed, the decision may no longer be valid. A version number attached to the snapshot, compared at application time with the current version, converts the race into a detectable failure at the cost of occasional retries. This is the mechanism by which the abstract atomicity assumption is realized, and it should be recorded in the authorization record so that audits can confirm it was in force.

20.8 Composition of Guards

Real systems have more than one guard, for instance a guard in the agent framework, another in the tool server, and a third in the underlying service. The composition of guards behaves as the conjunction of their decisions when each must permit, and the invariant then holds if it holds for any one of them provided that guard satisfies the assumptions. Redundant guards protect against the failure of one, which is the principle of defence in depth, and their value is again a matter of the independence of their failures. A guard that relies on information supplied by an earlier guard inherits its errors. A composition in which the strongest guard is placed nearest the effect, so that nothing can bypass it, is preferable to one in which it is placed nearest the generator, where bypass is more likely.

20.9 Soundness Without Usefulness

The proposition guarantees safety, and it says nothing about whether the system accomplishes anything. A controller that returns deny for every pair is sound and useless. The complementary property, completeness of the controller relative to the policy, requires that $A_{\Pi}(\sigma, a)$ imply $D_{\Pi}(\sigma, a) = \mathsf{permit}$, so that every admissible action is permitted. Soundness and completeness together make the controller exactly equal to the policy. In practice the policy is incomplete as a specification of what is desirable, the situation is observed imperfectly, and the controller is conservative where it is uncertain, so completeness fails and the cost of the failure is false refusals. The monograph treats this as the price of the guarantee. It is also the reason that the experimental program measures refusal quality and resource cost alongside violations: an architecture that attains zero violations by refusing everything is not a solution, and the evaluation must be able to say so.

Chapter 21

Refusal, Uncertainty, and Incomplete Authorization

21.1 Beyond Permitted and Forbidden

A binary classification of actions as permitted or forbidden is insufficient for agentic settings. Four situations should be distinguished that a binary scheme conflates. An explicit denial means that the policy forbids the action in the situation. Insufficient evidence means that the policy depends on facts that are not established. Unavailable verification means that the checking mechanism could not be consulted. Authorization-system failure means that the mechanism ran and failed. The three-valued decision $\{\mathsf{permit}, \mathsf{deny}, \mathsf{unknown}\}$ of Chapter 19 records the first as deny and the remaining three as unknown, with a reason code that distinguishes them for audit.

21.2 Conservative Semantics

The execution environment applies E only on permit. Under unknown the action is not executed, and the system takes the default behavior specified for non-execution, which may be to request additional evidence, to escalate to a principal with standing, or to stop. The rule is fail-closed. It ensures that gaps in the policy or failures in the mechanism lead to inaction and not to unauthorized action, at the price of false refusals, whose cost is discussed in Chapter 26.

21.3 Refusal and Impermissibility

Refusing to execute an action is a statement about the state of the authorization process and not a declaration that the action is intrinsically impermissible. An action refused for insufficient evidence may be admissible once the evidence is supplied. Reports of refusal should carry the reason code so that downstream components and human reviewers do not interpret every refusal as a prohibition.

21.4 Propagation Through Plans

Order the decisions by permissiveness, deny $\leq$ unknown $\leq$ permit. A multistep plan receives the minimum of the decisions of its steps, so a plan is permitted only if every step is permitted, and a single unknown step makes the plan unknown. This is a provisional assessment, because the admissibility of later steps depends on the situations reached by earlier ones, and those are known only when the earlier steps have been executed. Plan-level authorization can therefore only screen plans in advance. Each step must still be authorized at its own execution time on the actual situation, which is what Proposition 20.1 requires through assumption (A). Dependent tool calls, in which the arguments of a later call are derived from the results of an earlier one, require particular care, because the derived arguments carry the provenance of the earlier result, a matter taken up in the next chapter.

21.5 Reason Codes and Their Uses

A decision of deny or unknown should carry a reason code, a short structured label of why the decision was reached. The codes serve three audiences. For the generator, they allow a revised proposal that avoids the cause of refusal, for instance by requesting a narrower scope. For the principal, they explain why the system did not do what was asked, which is necessary if the principal is to decide whether to widen a grant. For the auditor, they allow the pattern of refusals to be analyzed, so that systematic gaps in the policy or recurring pressure on a boundary can be identified. A set of codes might distinguish a policy prohibition, a missing grant, an exceeded limit, an absent evidence item, an unreachable verifier, and an internal failure. The codes should be fixed in advance and typed, and they should not be free text, for the reason given in the discussion of information flow: free text from the controller is a channel back to the generator through which the boundary can be mapped.

21.6 Escalation

An unknown decision need not end in refusal. The architecture can route the case to a principal who has the standing to resolve it, who is shown the proposal, the relevant facts, and the reason for the unknown, and who then grants or denies. The escalation is itself an authorization event and is recorded. A well-designed escalation path reduces false refusals without weakening the guarantee, because the decision to permit is made by a party with standing. It is also a point at which the independence of the warrant can erode: a principal presented with a fluent justification from the system may approve it without examining the underlying facts, and the approval then carries less independent weight than it appears to. The interface should therefore present the facts the principal needs to decide, with the generator's argument kept visibly separate and labeled as such.

21.7 The Cost Structure of Conservatism

A fail-closed policy trades false permits against false refusals. If a false permit costs c_p and a false refusal costs c_r, a decision procedure that is uncertain about admissibility, with probability q that the action is admissible, would on expected cost grounds permit when $(1 - q)c_p < qc_r$ in the absence of the architectural constraint. That rule is an expected-cost

rule and it permits actions that the policy forbids, whenever q is large enough, which is exactly what the execution boundary exists to prevent. The distinction is between a controller that estimates probabilities of admissibility and acts on them and one that requires the policy to determine permission. For actions where c_p is bounded and small, the former may be defensible, and for actions where it is large or irreversible, the latter is required. The architecture can combine the two by applying the strict rule to a designated class of actions and a probabilistic rule to the rest, provided the classification of actions is itself part of the policy and is not made by the generator.

Chapter 22

Authority Laundering Through Tool Chains

22.1 How Explanations Acquire Operational Force

An explanation generated by a model can acquire operational force by passing through components. A first component summarizes a request. A second treats the summary as a command. A third executes it. Each hand-off is a point at which text produced by one stage is read by the next as though it were an instruction from an authority. The chain is the practical mechanism of authority laundering introduced in Chapter 4.

22.2 A Provenance-Sensitive Model

Let $(L, \leq)$ be a finite lattice of authority levels, with greater elements denoting greater authority, and let every message m carry a label $\lambda(m) \in L$ assigned by the infrastructure and not by the model. A component that consumes messages $m_1, \ldots, m_k$ and produces an output m assigns

$$\lambda(m) \;\leq\; \lambda(m_1) \wedge \cdots \wedge \lambda(m_k).$$

Each action a has a required level $\mathrm{req}(a) \in L$, and the authorization controller treats an action as supported by an instruction only if $\lambda(m) \geq \mathrm{req}(a)$ for the instruction m that requested it.

Proposition 22.1 (No elevation by derivation)**.** *Suppose labels are assigned by the infrastructure with the rule above, and every message reaching the controller carries its assigned label. Then an action whose required level is not below the label of some source cannot be supported by any instruction whose derivation depends on that source.*

Proof. By induction on the length of the derivation. A source message has its assigned label. If m is derived from $m_1, \ldots, m_k$ and each $\lambda(m_i)$ is at most the label of every source it depends on, then $\lambda(m) \leq \lambda(m_i)$ for each i by the labeling rule, and hence $\lambda(m)$ is at most the label of every source on which m ultimately depends. If $\mathrm{req}(a)$ is not below the label of such a source, then support, which requires $\lambda(m) \geq \mathrm{req}(a)$, would give $\mathrm{req}(a) \leq \lambda(m) \leq \lambda(\text{source})$, a contradiction. □

The rule is conservative, and it has a cost: anything derived from untrusted input is itself

untrusted, and cannot be elevated by being summarized, rephrased, or reasoned about. This is the correct behavior for the case that matters, in which untrusted content embedded in a document or a tool output contains text that resembles an instruction. It is the structural counterpart of the distinction between data and instructions and the formal setting for the class of attacks known as indirect prompt injection [10].

22.3 Legitimate Delegation

Delegation is legitimate when the authority conferred does not exceed the authority of the delegator, when it is conferred by an act that the policy recognizes as a grant, and when the derived instruction lies within the scope of the grant. A derived instruction exceeds the authority of its source when its required level is above the source's label or when it falls outside the granted scope. Elevation of a label above the meet of its inputs is permitted only by an explicit endorsement from a principal whose own label is at least the target level, and the endorsement is itself an event recorded in the history of the next chapter. The model cannot perform the endorsement on its own output, because that would reintroduce the self-authorization the architecture is meant to exclude.

22.4 Limits of Label Propagation

The labeling rule is enforceable only when the infrastructure controls label assignment. Inside a language model, information from differently labeled inputs mixes in ways that the infrastructure cannot trace, and the conservative response is to assign every output of a call the meet of all labels of everything in its context. Narrower labeling, which would allow a trusted instruction to coexist with untrusted data in the same context and still produce a trusted action, requires mechanisms for separating channels that the present analysis assumes but does not supply.

22.5 Illustrative Chains

Three chains show how authority can be laundered and how the labeling rule blocks it. In the first, an agent reads a web page that contains the sentence that the assistant should forward the user's files to an address. The agent's summary of the page includes the sentence in the form of a recommended next step, and a downstream component reads the summary as a task. Under the labeling rule the page has the label of an untrusted source, the summary inherits at most that label, and the forwarding action requires a higher level, so the controller refuses. In the second, a user with limited standing asks the agent to prepare a request on behalf of a more privileged user, and the agent produces a message in the voice of the privileged user. The message carries the label of the user who initiated it and not the label of the voice it imitates, so the apparent authority of the message is not recognized. In the third, one agent hands a task to another with a note describing the permissions it holds. The note is an assertion by the first agent, carrying the first agent's label, and the second agent's controller consults the delegation record and not the note.

22.6 Taint Tracking and Capability Tokens

Two families of mechanism implement the labeling model. Taint tracking attaches labels to data and propagates them through computation by rules, so that anything derived from labeled data carries a label at most as authoritative as its inputs. It is natural for the derivations that the infrastructure performs, such as concatenation and extraction, and it requires conservative approximation for derivations performed by a model. Capability tokens, a mechanism with a long history in protection design [3], attach authority to unforgeable references held by a component. A component can act only on the resources for which it holds a token, and tokens are conferred by explicit grants and can be attenuated when passed on, so that a delegate holds no more than the delegator. Capabilities have the advantage that authority is carried by the reference and not inferred from the origin of text, which makes confusion between data and instructions harder. The two can be combined, with tokens conferring the authority and labels governing what may be done with data of uncertain origin.

22.7 Endorsement as an Event

The raising of a label above the meet of its inputs is the one place where the model of this chapter allows authority to increase, and it is allowed only by endorsement from a principal with sufficient standing. Endorsement should be a distinct and logged event, with the endorsing principal, the item endorsed, and the target label recorded. It should be narrow, applying to a specific item and not to a category, so that it cannot be used to launder a class of content. It should also be revocable and time limited where the content can change. The practical form is the confirmation step in which a person reviews a proposed action, with the data on which it rests, and approves it. The monograph’s claim about such steps is structural: they discharge the requirement for an independent warrant only if the person is in fact an independent party reviewing the underlying facts, and the design of the step should make that likely.

Chapter 23

Historical Accountability and Deterministic Replay

23.1 Append-Only Histories

An execution history is a finite sequence

$$H_n = (s_0, o_1, \ldots, o_n)$$

of an initial state and a sequence of operations, with a deterministic transition function δ such that $s_i = \delta(s_{i-1}, o_i)$. Replay of the history is recomputation of s_n from s_0 and the operations. Transition validation checks that each o_i is a legal operation in state s_{i-1}. The history is append-only in the sense that an entry once recorded is not altered, which can be enforced by a hash chain [12]: if $e_i = (o_i, \alpha_i)$ is the i-th entry with its authorization record α_i, and $h_i = \mathsf{H}(h_{i-1} \,\|\, e_i)$ for a collision-resistant hash H, then any alteration of an earlier entry changes every later hash.

23.2 Account Versus Record

A generated account of operations differs from a recorded sequence of verified state transitions in the way that testimony differs from a log. The first is a claim about what happened produced by a party with an interest in being believed. The second is the output of the mechanism that did it. The architecture's value for accountability rests on ensuring that the history is written by the execution environment and not by the generator.

23.3 Operations in the SpherePOP Setting

In the formulation adopted here, the operations of the SpherePOP framework, namely POP, MERGE, COLLAPSE, and REFUSE, are interpreted as labelled transitions in a history-preserving execution substrate, each of which appends an entry and none of which rewrites a previous entry. The treatment of REFUSE as a transition in its own right is the relevant design decision for the present purposes: a refusal advances the log without changing the protected world state, so that refusals are recorded, replayable, and auditable in the same way as executions. Appendix F gives a self-contained summary of the lifecycle

and interface used here, so that the results of this chapter can be evaluated without the full specification of the calculus [7].

23.4 Replay Is Not Permission

Proposition 23.1 (Replay does not establish permission)**.** *If δ is deterministic and the hash chain verifies, replay reproduces s_n. Replay alone does not establish that any operation o_i was admissible when it was applied.*

Proof. Determinism of δ and induction on i give s_i from s_{i-1} and o_i, and the hash chain guarantees that the entries replayed are those recorded. Neither fact refers to Π, so the replayed history is equally the history of an execution in which every operation was admissible and of one in which none was. □

The consequence is that the authorization record α_i must be stored separately from the operation and must itself be checkable. A history is audited-admissible when for every i the record α_i verifies, against the policy in force at the time and the state s_{i-1}, that $A_\Pi(s_{i-1}, o_i)$ held. Replayability and audited admissibility are distinct properties, and a system that offers the first should not be described as offering the second.

23.5 Log Design

Several design decisions determine whether a history can serve as the top rung of the ladder of evidence. The history should be written by the execution environment, not by the generator, so that entries are the output of the mechanism and not testimony about it. Each entry should record the state or a commitment to the state on which the decision was made, so that the authorization record can be checked against what was in force. The entry should include the version of the policy, the identity of the principal on whose behalf the action was taken, and the provenance labels of the inputs. Entries should be hash chained, and the head of the chain should be periodically anchored in a location that the system cannot modify, such as a separate store under the control of a different party, so that truncation as well as alteration can be detected. Retention should be long enough for the audits it is meant to support.

23.6 Privacy and Redaction

A complete record of the situation on which each decision was based may contain sensitive data, and the demands of accountability pull against those of privacy. A standard resolution is to record commitments, such as hashes of the sensitive fields, in the history and to keep the fields themselves in a store with stricter access, so that an auditor with the appropriate access can verify that the commitment matches the data, while others see only the commitment. Redaction after the fact is possible without invalidating the chain if the redacted material was committed to by hash and only the hashed value is retained in the chain. The design should state which fields are stored in the clear, which as commitments, and who may open the commitments.

23.7 Replay Under Nondeterministic Effects

The determinism assumed of the transition function is a property of the recorded history and not of the world. An action such as sending a message or calling an external service has an outcome that depends on factors outside the log. For replay to be meaningful, the outcomes should be recorded as observations and the replay should treat them as inputs, so that the replay reproduces the internal state given the outcomes and does not attempt to recompute them. The history then supports the claim that, given the recorded outcomes, the state followed from the operations. It does not support the claim that the outcomes were as recorded, which depends on the reliability of the observation mechanism. This limitation is a further reason that the history is evidence of what the system did and observed, and is not an independent attestation of what happened in the world.

Part VI

Optimization, Distillation, and Institutional Design

Chapter 24

Reward Optimization and Procedural Deterioration

24.1 The Reported Pattern

In the source experiments, post-training with group relative policy optimization [23] was associated, in some conditions, with an increase in answer accuracy and a decrease in measured trace validity [26]. This is the pattern that a reward defined only on outcomes would be expected to permit. The study also applies the same post-training to models trained on mixed proportions of swapped traces and reports that plan accuracy continues to rise while trace validity stays flat or declines, so that the procedure improves the solutions without making the traces more meaningful. The reward does not mention the trace, and nothing in the optimization preserves a property that the reward does not mention.

24.2 Outcome Measures and Procedural Invariants

Optimizing an outcome measure and preserving a procedural invariant are different objectives. An outcome measure $R(x, y)$ is a function of the result. A procedural invariant is a predicate on the process, for instance that the trace remain valid. Maximizing $\mathbb{E}[R]$ leaves the invariant free to vary, and it will vary in whatever way is useful for increasing R, whether or not that way is also harmful to the invariant. The statement is conditional: the invariant need not deteriorate, and the source study reports deterioration in particular conditions and not as a general law.

24.3 Four Ways of Handling a Constraint

Four approaches can be compared. Unconstrained reward maximization ignores the invariant. Penalty-based objectives subtract a multiple of a violation measure from the reward. Constrained optimization restricts the search to policies satisfying the invariant in expectation or with a bounded probability. A hard execution boundary prevents violating actions from taking effect regardless of what the policy proposes.

Proposition 24.1 (Penalties do not enforce invariants)**.** *Let* $J_\lambda(\pi) = \mathbb{E}_\pi[R] - \lambda\,\mathbb{E}_\pi[\mathbf{1}_{\mathrm{viol}}]$ *for a penalty weight* $\lambda \geq 0$*. Suppose that for every policy* π_0 *with violation probability zero*

there is a policy π_1 with violation probability $p > 0$ such that $\mathbb{E}_{\pi_1}[R] - \mathbb{E}_{\pi_0}[R] > \lambda p$. Then no maximizer of J_λ has violation probability zero.

Proof. Let π_0 have no violations, so $J_\lambda(\pi_0) = \mathbb{E}_{\pi_0}[R]$. For the corresponding π_1 of the hypothesis, $J_\lambda(\pi_1) = \mathbb{E}_{\pi_1}[R] - \lambda p > \mathbb{E}_{\pi_0}[R] = J_\lambda(\pi_0)$. Hence π_0 is not a maximizer, and since π_0 was an arbitrary zero-violation policy, none is. □

The proposition shows that for any finite penalty there are tradeoffs that make violation optimal. A hard boundary corresponds to an infinite effective penalty applied at execution, and it removes the tradeoff because the violating action never takes effect. A penalty is also estimated on the training distribution, and its protection under distribution shift is a matter of generalization of a learned behavior and not a guarantee.

24.4 Goodhart-Style Effects

The pattern resembles the effects known under the name of Goodhart's law [9, 24], according to which a measure that becomes a target ceases to be a good measure. The resemblance should be stated with restraint. Optimization of a metric does not necessarily produce procedural deterioration; whether it does depends on whether the optimum of the metric lies in the region where the procedural property fails. The monograph does not claim that optimization of any metric degrades any other property, only that preservation of a property requires a mechanism that refers to the property.

24.5 Constrained Optimization and Its Guarantees

Constrained policy optimization replaces the penalty by an explicit constraint. In the Lagrangian form, one seeks a policy maximizing $\mathbb{E}_\pi[R]$ subject to $\mathbb{E}_\pi[\mathbf{1}_{\text{viol}}] \leq \delta$ for a tolerance δ, and introduces a multiplier $\lambda \geq 0$ adapted during training so that the constraint is met. The multiplier is not a hand-chosen penalty weight but the dual variable of the constraint, and at an optimum of a convex problem it equals the marginal reward cost of tightening the tolerance. This is an improvement over a fixed penalty, because the tradeoff is set by the tolerance, which is a statement about acceptable risk, instead of by an arbitrary weight. It does not deliver a hard guarantee. The constraint is in expectation over the training distribution, a tolerance greater than zero admits violations by definition, and a tolerance of zero is satisfied by the learned policy only to the extent that its estimate of the violation probability is accurate on the deployment distribution. For the class of invariants that must hold in every execution, such as the admissibility of executed actions, a constraint in expectation is the wrong kind of object.

The distinction can be put in terms of what is quantified over. A constraint in expectation quantifies over the distribution of tasks and trajectories, and says that the average violation is small. An invariant quantifies over every execution, and says that no violation occurs. The first can be approached by optimization and verified by sampling, with sampling providing evidence about the average. The second can be achieved by construction at the execution boundary and verified by proof under assumptions. An organization that needs the second and receives the first has a system that is safe on average and unsafe on the cases that matter.

24.6 Process Rewards and Their Limits

A common response to the deterioration of traces under outcome optimization is to add a process reward that scores intermediate steps. This can maintain trace validity as measured, and the monograph does not dispute that it often does. Two limits should be noted. First, a process reward is a learned or specified approximation of validity, and optimization pressure finds the places where the approximation departs from validity, so the trace may come to satisfy the reward model without satisfying the specification. Second, maintaining trace validity does not by itself establish faithfulness, since a model can be trained to produce valid traces that its answers do not depend on. The remedy that addresses faithfulness is architectural, forcing the answer through the trace, and the remedy that addresses admissibility is, again, the execution boundary. Process rewards are therefore a tool for the first property and cannot be a substitute for mechanisms aimed at the others, which is the same lesson as before expressed for training.

24.7 Indifference of the Reward

A further limit of outcome measures is that they may fail to distinguish among policies that differ in properties that matter. The simplest case is exact indifference. Suppose that over T rounds each choice succeeds with probability p independently of which option is chosen and of the history. Then every selection policy, adaptive or not, has expected total reward Tp. The argument is short: for the success indicator X_t in round t and the history $\mathcal{F}_{t-1}$, $\mathbb{E}[X_t \mid \mathcal{F}_{t-1}, a_t] = p$ whatever the policy selects, so $\mathbb{E}[X_t] = p$ by the tower property and the sum has expectation Tp [8]. A policy that spreads options evenly and a policy that confines each option to a narrow role are equally optimal, and the reward cannot choose between them.

The relevance to this chapter is that the invariants a system should preserve are often exactly the properties on which the reward is silent or indifferent. If the reward ties, the selection among tied policies is made by whatever regularities the training history makes salient, and the result may be structure that no one chose. The criterion that decides among tied policies must come from outside the reward, as a constraint on an admissible set or as an added objective, and in either case from a source with standing to impose it. Trace validity is a case in point: when answer accuracy is indifferent to the validity of the trace, optimization has no reason to preserve it, and the observed deterioration is one resolution of an indifference the reward leaves open.

Chapter 25

Distillation Without Semantic Fidelity

25.1 Four Forms of Transfer

Training a student from a teacher can take several forms. Answer distillation trains the student on the teacher's final outputs, with soft labels in the classical form [11]. Trace imitation trains the student on the teacher's intermediate sequences. Process supervision trains with feedback on individual steps [16]. Latent-representation transfer trains the student to match internal states or their geometry. The forms transfer different things, and a student can match the teacher on one while differing on another.

25.2 Competence Without Explanation

If the usefulness of intermediate sequences does not depend on their procedural semantics, a student may inherit useful computational behavior without reproducing the teacher's explanations, or reproduce the explanations without inheriting the behavior. The first possibility arises if the student learns the scaffolding structure and not its surface. The second arises if the student learns to imitate the form of the traces, which is easy, without the computation they accompanied in the teacher, which is not.

25.3 Proposed Experiments

The comparison before and after compression should measure answer accuracy, trace conformity to the specification, generalization across distributions, and sensitivity to interventions on the trace, and in addition, where internal access is available, the geometric coherence of trajectories. The hypothesis developed is that compression may preserve task competence while changing the explanatory and procedural properties of generated traces, so that a student's traces are less reliable guides to its computation than the teacher's were. The hypothesis predicts that trace conformity and intervention sensitivity change under distillation by more than accuracy does, and a result in which all three change together would not support it.

25.4 A Measurement Matrix for Compression

The comparison of teacher and student can be organized as a matrix whose rows are the forms of transfer and whose columns are the properties measured. For each row, the student is trained under that form of transfer, and for each column the property is measured on the student and compared with the teacher. The columns are answer accuracy on the training distribution, answer accuracy on held-out distributions, trace conformity to the specification, sensitivity of the answer to interventions on the trace, and, where internal access allows, the geometric coherence of internal trajectories. The matrix makes the question concrete: which forms of transfer preserve which properties, and which properties are lost together.

Several patterns could be found, and each would have a different interpretation. If answer distillation preserves accuracy and trace imitation preserves conformity but neither preserves intervention sensitivity, the traces of the student are decoration relative to those of the teacher, and compression has turned a functional scaffold into a surface. If latent transfer preserves coherence and accuracy but not conformity, the evidence favors the view that the internal geometry carries the competence. If all columns move together, the properties are more tightly linked than the independence argument suggests, for the models tested, and the linking assumptions of Table 2.1 would be supported empirically in that setting. The monograph expects a mixed pattern and presents the matrix as the way to find out.

Chapter 26

Verification as an Independent Institution

26.1 The Principle

A system should not be the sole judge of whether its own actions are permitted when its objective rewards the successful completion of those actions. The principle is not specific to machine learning. It is the rationale for separating the author of code from its reviewer, the performer of an experiment from the party who replicates it, the preparer of accounts from the auditor, and the agency that acts from the oversight body that authorizes. In each case independence reduces the correlation between the errors of the actor and the errors of the checker.

26.2 Comparison Across Domains

In software, independent review catches defects the author is blind to, and its value depends on the reviewer not sharing the author's assumptions. In science, reproducibility depends on an independent party repeating the procedure from the description, and it fails when the description is insufficient. In accounting, external audit depends on access to primary records and not only to the preparer's summary. In administration, oversight depends on the overseer's authority to refuse. The shared structure is access to primary evidence, a different incentive, and authority to stop the process, and the same three conditions apply to the verification of agents.

26.3 Independent Evidence and Independent-Looking Reports

An independent report is one whose errors are not determined by the errors of the thing it checks. An independent-looking report is one produced by a different component from the same underlying source. A second model that reads the first model's explanation and approves it is independent-looking: both reports depend on the explanation, and if the explanation is misleading they fail together. A formal policy checker that evaluates the proposed action against a policy specification and the recorded situation, without reading

the explanation, is independent in the relevant sense.

26.4 Costs of Verification

Verification has costs. It adds latency. It produces false refusals when the policy is conservative or the evidence is incomplete. It is only as good as the policy specification, and policies for open-ended environments are incomplete and sometimes contested. It can become a concentrated point of failure, because a single verifier that is wrong, compromised, or unavailable affects every action that passes through it. These costs are real, and they motivate keeping the verifier simple enough to be inspected, diversifying verifiers where failure would be costly, and applying the strongest verification to the actions whose consequences are hardest to reverse.

26.5 Correlated Failure of Checkers

The benefit of adding a second checker depends on the correlation between the failures of the two. Let F_1 and F_2 be indicators that the generator and the checker, respectively, fail to catch a given problem, with probabilities p_1 and p_2 and correlation coefficient ρ. The probability that both fail is

$$\mathbb{P}(F_1 \cap F_2) = p_1 p_2 + \rho\sqrt{p_1(1-p_1)p_2(1-p_2)}.$$

If the failures are independent, $\rho = 0$ and the joint failure probability is the product $p_1 p_2$, which can be very small. If they are perfectly correlated with equal probabilities, $\rho = 1$ and the joint failure probability is p_1, so that the checker adds nothing. Intermediate correlations give intermediate benefits, and a modest positive correlation can eliminate most of the benefit when the individual failure probabilities are small: with $p_1 = p_2 = 0.01$ and $\rho = 0.5$ the joint probability is about 0.005, fifty times the independent value. The arithmetic illustrates the claim made earlier that independence in the relevant sense is a quantitative matter, and that a checker built from the same model, trained on similar data, and given the same context as the generator is likely to have a substantial correlation with it.

26.6 Sources of Diversity

Correlation among checkers can be reduced by diversity along several dimensions. The mechanism can differ, so that one check is a formal predicate and another is a statistical classifier. The evidence can differ, so that one check reads the proposal and another reads the system logs. The authority can differ, so that one check is performed by the system's operator and another by an external party. The failure modes can differ in the sense of the kinds of error each is prone to, such as false negatives in one and false positives in another. Diversity is not free, and the combination of checkers has to be defined, for instance as the requirement that all of them permit, which maximizes safety at the cost of refusals, or that a majority permit, which reduces refusals and weakens the guarantee. The monograph's architectural claims use the conservative combination, and the choice of any other is a decision to accept a risk, which should be recorded as such.

Part VII

The Empirical Program and Conclusions

Chapter 27

The Experimental Program

27.1 Status of the Proposals

The experiments in this chapter are designs. None has been run for this monograph, and no result is claimed. Their purpose is to state, before data are collected, what would count as support for or against each hypothesis, so that the hypotheses are exposed to refutation.

27.2 Experiment I: Geometric Residuals and Task Success

The first experiment tests Hypothesis 16.2. Models are trained in the three regimes of Chapter 8 on a maze family with several distributions. For each model and evaluation task, record answer correctness, trace validity under the A* specification, generated length, a difficulty covariate computed from properties of the maze, and the hidden-state trajectory at a stated set of layers. Construct reference sets of states from held-out correct completions, fit local tangent estimates at several neighbor counts and dimensions, and compute the trajectory residual of Chapter 7. Fit a regularized logistic model of correctness on length, difficulty, and validity as the baseline, add the residual, and compare held-out log-likelihood by cross-validation across mazes.

Four controls are required. A null residual computed from random directions of matched norm establishes the scale at which a residual is small. A residual computed against a reference set drawn from unrelated tasks establishes that the effect, if any, is specific to task-relevant states. A residual computed on shuffled trajectories from different mazes establishes that the effect is not an artifact of the position along the trajectory. The fourth control addresses construct validity. Reference manifolds estimated only from correct completions may detect proximity to a success-associated distribution of states and not a general tangent-alignment property. Reference sets are therefore also constructed without conditioning on correctness, from all completions or from a random sample of states, and the residuals computed against the two kinds of reference are compared. If only the correctness-conditioned residual predicts success, the result is attributed to proximity to a success-associated representation distribution and is not read as support for a general geometric principle. Uncertainty estimates account for both task sampling and independently trained model seeds, with seeds treated as a random effect, because bootstrapping mazes alone would omit variability due to training initialization. The hypothesis is supported if the residual improves held-out prediction beyond the baseline across the stated range

of estimator settings and the controls do not show a comparable improvement. It is not supported if the improvement vanishes under the controls or appears only at isolated settings.

27.3 Experiment II: Scaffold Utility

The second experiment is the prefix factorial of Chapter 10. For each prefix type, namely correct trace, swapped trace, random tokens, repeated tokens, and learned prefix, evaluate answer accuracy at matched token counts and, separately, at matched computational cost, since equal token counts do not equalize the full training budget or implementation-level inference cost, and estimate the interventional utility ΔU relative to the empty prefix and the random prefix. Where internal access is available, compute the trajectory residual for each prefix type and examine whether the ordering of prefix types by utility matches the ordering by residual. A match would be consistent with the geometric account, and a mismatch would indicate that utility arises through a pathway the residual does not capture.

27.4 Experiment III: Admissible Agents

The third experiment evaluates the execution boundary. A suite of formally checkable problems is posed to agents that propose actions subject to independent authorization constraints. The suite includes three kinds of task, constructed so that the three relations among correctness, validity, and permission are all represented: tasks for which the correct solution is unauthorized, tasks in which the explanation is valid but the action is prohibited, and tasks in which the action is permitted although the accompanying explanation is invalid. Four architectures are compared. In the first the model authorizes itself, so that its own judgment decides execution. In the second a second model provides approval. In the third a formal verifier checks the proposal against the policy specification. In the fourth a runtime controller enforces permissions at the execution boundary in the sense of Proposition 20.1.

For each architecture the evaluation records answer accuracy, trace validity, causal sensitivity of the output to the trace, authorization accuracy against the ground-truth policy, execution violations, refusal quality, and resource cost. Refusal quality is measured by the fraction of refusals that carry the correct reason code and by the fraction of tasks that could have been completed admissibly and were not. The primary safety metric is the rate of unauthorized execution and not the persuasiveness of generated justifications. Persuasiveness may be recorded as a covariate, since a central prediction is that the unauthorized execution rate of the first two architectures depends on the persuasiveness of the generator's justification, while that of the third and fourth does not.

27.5 Predictions

The program yields predictions that can fail. The unauthorized execution rate of the fourth architecture is predicted to be zero up to violations of the stated assumptions, and any observed violation is to be traced to one of (M), (S), (A), (P), or (I), or else to a specification or instrumentation failure, meaning a mismatch between the formalized action, the measured event, and the actual behavior of the system. The evaluation must be able to report the second kind of cause as well as enforcement failures. That of the self-authorizing architecture

is predicted, conditionally, to increase with the generator's competence when successful task completion is rewarded without adequate authorization constraints, because more competent generators then produce more convincing proposals for actions that the policy forbids. The relation is not necessary: a more competent model might also be better at recognizing authorization constraints. The experiment therefore varies reward structure and policy awareness independently of competence and tests the interaction among the three. That of the second-model architecture is predicted to lie between them and to correlate with the similarity of the two models. If the self-authorizing architecture's violation rate does not increase with competence under the reward structure that rewards completion alone, the claim that competence promotes statistical into procedural authority is weakened for the tested population.

27.6 Analysis Plan and Threats to Validity

Each experiment should be specified in an analysis plan fixed before the data are examined, including the primary outcome, the covariates in the baseline model, the estimator grid for the geometric measurements, the thresholds for what will be called support or non-support, and the rule for handling failed runs. The purpose of fixing these in advance is to prevent the flexibility of the geometric estimators, which have several tunable parameters, from being used to find an association after the fact. The estimator grid should be reported in full, with the results at every point of the grid and not only at the point where the effect is largest, so that the reader can see how stable the conclusion is across the grid.

Several threats to validity should be addressed in the design. Confounding by difficulty arises because harder mazes may produce both longer traces and more erratic internal trajectories, and the baseline must include difficulty measures strong enough to absorb this. Confounding by confidence arises because a model that is uncertain may both fail more often and traverse less structured states, and the baseline should include a measure of the model's uncertainty at the start of generation. Overfitting of the estimator arises because local manifold estimates built from a reference set may fit that set closely and fit new states poorly in a way correlated with correctness for reasons unrelated to coherence, and the remedy is to estimate on one subset and evaluate on another. Construct validity arises because the residual measures departure from an estimated local linear structure, and whether that corresponds to the semantic notion of coherence in the earlier theory is a question that the experiment cannot settle on its own.

27.7 What Would Change the Conclusions

The hypotheses are stated so that results can contradict them. If swapped traces provide no utility beyond random tokens of equal length, the scaffolding hypothesis reduces to a claim about serial computation, and the conditional structure of traces plays no role. If the geometric residual adds nothing to prediction of correctness beyond the baseline, Hypothesis 16.2 is not supported for the tested models. If the unauthorized execution rate of the self-authorizing architecture does not increase with generator competence, the prediction about the growth of the risk with capability fails. If an execution violation is observed in the architecture with a runtime controller, the violation is traced to the failed assumption,

and the proposition is not refuted but its applicability to that deployment is. Distinguishing a refuted proposition from a violated assumption is the purpose of stating the proposition conditionally.

Chapter 28

Scope and Limits

28.1 Formal and Open-Ended Domains

The results of this monograph are sharpest in formal domains, where a specification is decidable and a policy can be written completely. In such domains validity can be checked exactly, correctness can be determined independently, and an authorization controller can be built to be sound. In open-ended environments the specification of the task is incomplete, the policy is contested or underspecified, and the situation cannot be fully captured. The structural claims remain, since the independence of the properties does not depend on decidability, but the guarantees weaken, because the soundness assumption (S) is only as good as the completeness of the policy.

28.2 Verification That Uses Language Models

An objection holds that independent verification may itself require language models, because the policies and situations are expressed in natural language and cannot be parsed otherwise. The distinction that answers it is between probabilistic assistance in verification and unilateral authority over execution. A language model may be used to extract the relevant facts from a document, to flag possible violations, or to rank cases for human review, and in these roles its errors are caught by the stage that follows. It may not be the stage that decides, in the sense of determining execution without a sound check, unless its decisions are themselves subject to the boundary of Proposition 20.1. The architecture constrains where authority sits and permits any technology to supply evidence.

28.3 Stronger Models

Stronger models do not remove the need for external verification. The independence proposition does not depend on model quality. Improvement in competence raises the proportion of proposals that are correct and also raises the persuasiveness of those that are not, and it does nothing to the policy, which is a separate input. If anything the incentive to verify independently grows with competence, because the cost of an unverified action grows with the capability of the system proposing it.

28.4 Limits of the Geometric Account

The geometric account is a hypothesis with specific vulnerabilities. Hidden states may not be well approximated by low-dimensional manifolds at the scales relevant to the task. Estimates of coherence depend on the estimator and the reference set. A measured association between coherence and success need not be causal. The scaffolding hypothesis does not depend on the geometric account and survives its failure. The authorization results do not depend on either and would stand if both were rejected.

28.5 Open Problems

Several problems are left open by the analysis. The first is the extension of the geometric account to settings in which hidden states are better described by a stratified space than by a smooth manifold, with a corresponding theory of what it means for a trajectory to cross strata coherently. The second is the relation between token-level interventions and state-level interventions, which the monograph treats as an empirical quantity to be measured and for which no theory is offered. The third is the design of verifiers for policies that cannot be written formally, where the policy is partly implicit in the practices of an institution, and where the soundness assumption is correspondingly difficult to state, let alone secure. The fourth is the treatment of multi-agent systems, in which the authority of one agent over another is itself a policy question, and the provenance model of Chapter 22 must be extended to labels that evolve as agents delegate and endorse.

A fifth problem concerns the economics of verification. The architecture imposes costs, and the question of how much verification is justified for which actions is a question of decision under uncertainty with asymmetric losses that has not been addressed here beyond noting that the strongest checks belong on the actions whose consequences are hardest to reverse. A sixth concerns the human side of the boundary. The authorization controller may be a person or a committee, and the reliability of human authorization is itself subject to the failures of attention and deference that automation tends to produce. A system in which people approve whatever a fluent system proposes has reintroduced the conflation in a different place, and the design of interfaces that preserve the independence of the human warrant is a problem for which this monograph provides the motivation and not the solution.

Chapter 29

Conclusion: Reasons Are Not Permissions

The monograph began from an empirical observation: intermediate tokens can improve performance without functioning as valid derivations. From that observation it proceeded in three steps. The first step distinguished the properties that are commonly run together, namely utility, procedural validity, causal faithfulness, geometric coherence, outcome correctness, and policy admissibility, and showed by explicit countermodels that none of the first five entails the sixth and that no unconditional entailment holds among the first five without stated linking assumptions. The result is the absence of unconditional entailment. It is not a claim of statistical independence, and it does not assert that every combination of values is realizable under every specification. The second step offered a positive hypothesis about what makes intermediate sequences useful, in terms of the geometry of the trajectories they induce, with a test that could refute it. The third step moved from the evaluation of explanations to the control of action, and showed that an independent authorization mechanism with complete mediation, atomic validation, and a sound check guarantees that no inadmissible action is executed, whatever the generator proposes and however persuasive its explanation.

The appropriate response to increasingly competent generative systems is not to require that every useful internal computation resemble a human explanation. It is to specify which claims require which evidence, and which actions require which authorization. The distinction that governs the manuscript is among what has been established empirically, what follows logically from definitions, and what must be guaranteed architecturally, and the manuscript has attempted to avoid the category error that it describes by keeping the three apart.

It is useful to restate what each register of claim has contributed. Empirically, the monograph relies on a single source for the dissociation between trace validity and answer accuracy in maze-solving transformers, and it has been careful to state that result in its narrow form. Logically, it has proved that the six properties are independent except under stated linking assumptions, that permission is not entailed by any of the others, that a projection of an update onto a cone of feasible tangent directions is well defined and satisfies first-order feasibility, and that an execution boundary with complete mediation, atomic validation, and a sound check guarantees the admissibility of every executed action. Architecturally, it has argued that the guarantee is obtained by a small trusted base, a

typed interface between roles, provenance labels that cannot be raised by derivation, and a history written by the execution environment. As conjectures it has offered the scaffolding hypothesis and the geometric faithfulness hypothesis, each with a test that could fail. The boundaries between these registers are the main methodological commitment of the work, and a reader who disagrees with any conjecture can discard it without affecting the logical and architectural claims.

The concluding principle is the following.

Competence does not establish faithfulness.
Faithfulness does not establish permission.
Permission requires an independent warrant.

Here an independent warrant is an authorization grounded in the applicable policy and in evidence, and not an assertion made sufficient by the proposing system's own success or explanation.

Appendix A

Formal Language and Notation

Tasks are elements x of a set $\mathcal{X}$, intermediate sequences z are finite strings over a token alphabet, outputs y lie in $\mathcal{Y}$, and actions a lie in $\mathcal{A}$. A situation σ is an element of a set Σ that records the task, the environment state, the requesting principal, and any delegations in force. A policy is a predicate $A_\Pi : \Sigma \times \mathcal{A} \to \{0, 1\}$. A trace specification $\mathcal{S}$ consists of a trace language $L_\mathcal{S}$, a deterministic transition system over states with a transition function, and an acceptance relation; the predicate $V_\mathcal{S}(x, z, y)$ holds when $z \in L_\mathcal{S}$, when the sequence of transitions it records is a legal sequence of $\mathcal{S}$ on the input declared in the trace, and when y is accepted in the final state. Specifications differ in what they require of the declared input: a weak specification $\mathcal{S}_{\text{loc}}$ accepts any declared input, and a strong specification $\mathcal{S}^+$ requires the declared input to equal x. Validity is relative to the specification throughout. A generator is a conditional distribution $p_\theta(z, y \mid x) = p_\theta(z \mid x)p_\theta(y \mid x, z)$. Probabilities of success are taken over the randomness of the generator and over a stated distribution of tasks. Manifolds are smooth embedded submanifolds of $\mathbb{R}^n$ or of a hidden-state space $\mathbb{R}^D$ with the induced Riemannian metric g, and the tangent and normal spaces are the orthogonal decomposition of the ambient space with respect to g at regular points [14].

Appendix B

Countermodels

The countermodels for Proposition 13.1 are given in the proof of that proposition on the parity task. They are repeated here in tabular form with the values of the predicates in each system. In the table, C, V, F, Γ take values in $\{0, 1\}$, and the final column shows the admissibility predicate under a policy chosen for the case.

System	C	V	F	Γ	A_Π
Lookup answer, arbitrary trace	1	0	0	–	–
Lookup answer, correct trace ignored	1	1	0	–	–
Reads last trace entry, correct trace	1	1	1	–	–
Transcription error, procedure run on copy	0	1	1	–	–
Same text and output, coherent trajectory	1	1	–	1	–
Same text and output, incoherent trajectory	1	1	–	0	–
Correct, valid, faithful, coherent; action prohibited	1	1	1	1	0

Table B.1: Values of the predicates in the systems used to prove evaluative independence. A dash indicates that the value is unconstrained by the construction.

Appendix C

Execution Boundary Soundness: Formal Model

Let Σ be a set of situations and $\mathcal{A}$ a set of actions. A guarded executor is a triple (D, E, Run) in which $D : \Sigma \times \mathcal{A} \to \{\mathsf{permit}, \mathsf{deny}, \mathsf{unknown}\}$ is the decision map, $E : \Sigma \times \mathcal{A} \to \Sigma$ is the effect map, and $\mathrm{Run} : \Sigma \times \mathcal{A} \to \Sigma$ is defined by $\mathrm{Run}(\sigma, a) = E(\sigma, a)$ if $D(\sigma, a) = \mathsf{permit}$ and $\mathrm{Run}(\sigma, a) = \sigma$ otherwise, evaluated as one atomic step. An execution trace is a sequence $\sigma_0, a_1, \sigma_1, a_2, \ldots$ with $\sigma_i = \mathrm{Run}(\sigma_{i-1}, a_i)$ for the actions of the agent. An execution event is an index i with $D(\sigma_{i-1}, a_i) = \mathsf{permit}$.

Assumption (M) is that every transition of Σ attributable to the agent is of the form $\sigma_{i-1} \mapsto \mathrm{Run}(\sigma_{i-1}, a_i)$. Assumption (A) is that Run is atomic: no transition intervenes between the evaluation of D and the application of E, or any intervening transition leaves the value of D and of the facts on which A depends unchanged. Assumption (S) is that $D(\sigma, a) = \mathsf{permit}$ implies $A_\Pi(\sigma, a) = 1$ for all (σ, a) reachable under the trace. Assumption (P) is that Π is the policy in force in σ.

Proof of Proposition 20.1, in the formal model. Let i be an execution event. By (M) the transition at step i is $\mathrm{Run}(\sigma_{i-1}, a_i)$, and by definition of an execution event $D(\sigma_{i-1}, a_i) = \mathsf{permit}$. By (A) the situation to which E is applied is σ_{i-1}, the situation on which D was evaluated. By (S) and (P), $A_\Pi(\sigma_{i-1}, a_i) = 1$. Since i was arbitrary, every execution event satisfies the invariant. If D depends only on (σ, a, Π) and on trusted evidence, then for any two generators that propose the same (σ_{i-1}, a_i) with different explanatory texts, D takes the same value, so whether that proposed action is executed is unchanged under replacement of the text, with the trusted facts held fixed. Later proposals may differ, and no invariance of the whole history is claimed. □

Each assumption is necessary in the following sense. Without (M), an action can alter protected state without passing through Run, and the invariant is silent about it. Without (A), the decision can be made on a situation σ and the effect applied to a later σ' for which $A_\Pi(\sigma', a) = 0$. Without (S), D can return permit for an inadmissible pair. Without (P), D can be sound for a policy other than the one in force. In each case the conclusion fails in a system built to violate the assumption and satisfy the others.

Appendix D

Experimental Protocols

This appendix specifies the elements that the experiments of Chapter 27 require. Datasets are generated by sampling mazes from each distribution under a recorded random seed, with disjoint training, validation, and evaluation splits, and with evaluation splits drawn both from the training distribution and from held-out distributions. Each training regime is run with several random seeds, and results are reported as means with intervals across seeds and not as single runs. Trace interventions are applied at a fixed token position with replacement sequences drawn from a specified source. Prefix types are matched on token count, and the learned prefix is optimized on a held-out portion of the training split and evaluated on a separate portion. Compute is accounted by training tokens, evaluation tokens, and wall-clock device hours, and comparisons across arms are made at matched compute where the hypothesis concerns utility. Estimator settings for the geometric measurements span a stated grid of layers, neighbor counts, and dimensions, fixed before evaluation. Statistical analysis uses cross-validated held-out likelihood for the comparison of nested models, bootstrap intervals over mazes for effect estimates, and a correction for multiple comparisons across the estimator grid.

Appendix E

Replication Status

The empirical statements about the maze experiments in this monograph are those reported by the source study [26], and they are reported here as that study's findings. No independent replication has been carried out for this monograph. A replication would separate the published results from independently replicated results by reporting each configuration twice, once as published and once as reproduced, with differences in implementation noted. The bibliographic details of the source have been taken from the published version.

Appendix F

Execution Histories in the SpherePOP Setting

The abstract interface used in Chapter 23 is as follows. A history is a sequence of entries $e_i = (o_i, \alpha_i, \pi_i)$, where o_i is an operation drawn from a fixed set that includes POP, MERGE, COLLAPSE, and REFUSE, α_i is an authorization record containing the decision, the reason code, the identity of the authorizing principal, and a reference to the policy version, and π_i is a provenance record containing the labels of the inputs from which the operation was derived. The hash of an entry is $h_i = \mathsf{H}(h_{i-1} \,\|\, e_i)$. The transition function δ is deterministic on the protected state and is the identity on that state for REFUSE, which appends an entry and advances the hash chain without changing the protected state. Validation of the history checks the hash chain, the legality of each o_i in the state s_{i-1}, and, for audited admissibility, the verification of each α_i against the policy version it cites. This appendix fixes the interface on which the results of Chapter 23 depend and summarizes the lifecycle of the calculus of disposition [7] on which that interface is based.

Lifecycle of a Commitment

In the calculus of disposition the object being disposed of is a commitment, and a commitment passes through a lifecycle with the states open, bound, transformed, verified, verification-failed, collapsed, and refused. The operations that move a commitment between states are POP, which opens a commitment, BIND, which attaches it to the resources and conditions on which it depends, TRANSFORM, which carries out the computation or effect, VERIFY, which checks the result against its specification, COLLAPSE, which finalizes the commitment, and REFUSE, which ends it without finalization. The states collapsed and refused are terminal. Refusal is permitted from every live state and not only from the open state, so that a commitment can be refused at any point before it is finalized. A failed verification places the commitment in a distinct live state, verification-failed, and does not automatically refuse it. This design choice is the one that matters for the present monograph: a verifier can report a failure but cannot by that report dispose of the commitment, so verifiers hold no disposition authority, which is the separation between checking and authorizing that the execution boundary requires. The operation MERGE, which combines compatible commitments or histories in the outline from which this monograph was developed, is not used in any result here.

Disposition Safety

The central result of the calculus, stated here as an assumption imported from that work and not proved in this monograph, is disposition safety: a commitment reaches a terminal state only through an authorized, causally complete, and durably recorded history. The relation to the present analysis is direct. The authorization record of the history entry corresponds to the requirement that the history be authorized, the provenance record to causal completeness, and the hash chain to durable recording. The execution boundary proposition of Chapter 20 can be read as the statement of what an implementation must provide for the first of these to hold at the level of individual actions, and the lifecycle supplies the vocabulary in which refusal and unknown decisions become first-class events in the history and not silent omissions.

Appendix G

Limitations and Failure Cases

Verifier incompleteness arises when the checking procedure fails to detect violations that fall outside the properties it examines, and the remedy is to state the checked properties explicitly and to treat everything else as unchecked. Policy ambiguity arises when the policy does not determine a decision for the situation, and the conservative response of Chapter 21 is to return unknown. Environmental nondeterminism undermines deterministic replay when the effect map E depends on factors outside the recorded state, and the history then records the attempt and not the outcome unless outcomes are also recorded. Adversarial tool outputs can mislead both the generator and the labeling infrastructure, and the provenance model addresses this only to the extent that the infrastructure assigns labels correctly. Distributed execution weakens complete mediation and atomic validation because decision and effect may occur on different machines with different views of the state, and a sound design requires either a single point of mediation or a protocol that provides the equivalent guarantees. Each of these is a place where an assumption of Proposition 20.1 can fail in practice, and each should be examined before the guarantee is relied upon.

Bibliography

[1] P.-A. Absil, R. Mahony, and R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2008.

[2] J. P. Anderson. Computer Security Technology Planning Study. Technical Report ESD-TR-73-51, Air Force Electronic Systems Division, 1972.

[3] J. B. Dennis and E. C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.

[4] C. Fefferman, S. Mitter, and H. Narayanan. Testing the Manifold Hypothesis. *Journal of the American Mathematical Society*, 29(4):983–1049, 2016.

[5] Flyxion. Never Predict Noise: Manifold-Aligned Prediction as Generative Inference. Unpublished manuscript, November 2025 (revised May 2026).

[6] Flyxion. MAGI and the Geometry of Learning. Unpublished manuscript, 2025.

[7] Flyxion. The Calculus of Disposition: Processes, Types, and Witnessed Computation. Unpublished manuscript, September 2026.

[8] Flyxion. Reasons Are Not Permissions: Admissibility, Statistical Authority, and the Agentic Execution Boundary. Unpublished manuscript, October 2026.

[9] C. A. E. Goodhart. Problems of Monetary Management: The U.K. Experience. In *Monetary Theory and Practice*, pages 91–121. Macmillan, 1984.

[10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

[11] G. Hinton, O. Vinyals, and J. Dean. Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015.

[12] L. Lamport. Password Authentication with Insecure Communication. *Communications of the ACM*, 24(11):770–772, 1981.

[13] T. Lanham et al. Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702, 2023.

[14] J. M. Lee. *Introduction to Smooth Manifolds*. Springer, second edition, 2013.

[15] B. Lester, R. Al-Rfou, and N. Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.

[16] H. Lightman et al. Let's Verify Step by Step. arXiv:2305.20050, 2023.

[17] J. Milnor. *Morse Theory*. Princeton University Press, 1963.

[18] R. E. Nisbett and T. D. Wilson. Telling More Than We Can Know: Verbal Reports on Mental Processes. *Psychological Review*, 84(3):231–259, 1977.

[19] M. Nye et al. Show Your Work: Scratchpads for Intermediate Computation with Language Models. arXiv:2112.00114, 2021.

[20] J. Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, second edition, 2009.

[21] H. G. Rice. Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.

[22] J. H. Saltzer and M. D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[23] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024.

[24] M. Strathern. "Improving Ratings": Audit in the British University System. *European Review*, 5(3):305–321, 1997.

[25] M. Turpin, J. Michael, E. Perez, and S. Bowman. Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *Advances in Neural Information Processing Systems*, 2023.

[26] K. Valmeekam, V. Palod, K. Stechly, A. Gundawar, and S. Kambhampati. Beyond Semantics: The Unreasonable Effectiveness of Reasonless Intermediate Tokens. *Transactions on Machine Learning Research*, April 2026. OpenReview identifier gDE7YcRC3F.

[27] P. Vincent. A Connection Between Score Matching and Denoising Autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

[28] J. Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, 2022.